PromiseTracker v3 system specification

Draft 3.0.0-draft.3, 9 November 2026. Prepared for team review.

Comments by: *[date]*. **Decision meeting:** *[date]*.

This single file combines all twelve documents of the specification. The repository keeps them separate under `docs/v3/`, and a browsable version is at https://claude.ai/artifact/YEPF121pBLxJ428AxF17fA

Cross-references appear as plain section numbers, such as §4.3, because the originals link between files. Use the contents below, or regenerate a linked table of contents in Google Docs with Insert, then Table of contents.

Contents

- Review brief
- Overview
- 1. Product
- 2. Architecture
- 3. Data model
- 4. Governance
- 5. Pipeline
- 6. Plugins
- 7. Public site
- 8. Security and operations
- 9. Launch
- 10. Roadmap

Read the review brief first. It says which five decisions are worth arguing about and which six questions need an answer, and it tells each reader how much of the rest to read.

Review brief

The spec is detailed on purpose. The expensive mistakes in a system like this get made in the first month and paid for over three years. You are not being asked to read all of it.

What we are asking for

Three things, in order of how much they are worth.

1. **Argue with the five decisions in §A.** They are cheap to change this week and expensive to change after Phase 1.
2. **Answer the question next to your name in §B.** Three of the six block work, and one blocks launch.
3. **Tell us what is missing** about how you actually work. Specifics beat approval.

We are not asking about wording, visual design or the phase dates. Those change cheaply later.

How much to read

If you are	Read	Time
Anyone, minimum	This brief	10 min
Programme or editorial lead	§1 Product, §4 Governance, §9 Launch	40 min
Fact-checker or editor	§1.5 Core flows, §5 Pipeline, §3.4 and §3.5 statuses and updates	30 min
Developer	§2 Architecture, §3 Data model, §6 Plugins	90 min
Ops or security	§8 Security and operations, §2.4 Mode matrix	20 min

If you read one page beyond this brief, read §1 Product.

The short version

Two ideas carry the design.

- **Everything is a plugin, including the core.** A thin kernel defines five interfaces, a core plugin supplies the whole domain, and Superdesk, each AI vendor, Tika and Airtable are replaceable adapters behind those interfaces.

- **One site, many countries.** No multi-tenancy. A country is a path segment, as in `promisetracker.org/ke/politicians/william-ruto`, not a deployment.

The test v3 has to pass: the system runs end to end with no external service at all. No Superdesk, no AI key, no Airtable, no Meedan Check. Today v2 stops working if you remove Airtable or Check, which is why the last row of §1.7 counts required external services and sets the target at zero.

Nothing needs migrating. The product has never gone live, so §9 is a launch plan.

A. Five decisions worth arguing about

The spec index records eighteen decisions. These five are where being wrong costs the most.

A1. One pipeline for four claim types

Manifesto promises, policy pledges, budget commitments and public statements all run through one pipeline and one review queue.

- **Gain.** No second system later for the things that are not quite promises.
- **Cost.** Every public page has to read sensibly for four types, and "kept" or "broken" language does not fit a one-off statement.
- **Changes our mind.** The newsroom only ever wants manifesto promises. Then `type` disappears and §3.3 and §7 get simpler.

A2. No multi-tenancy. Country is a path segment

- **Gain.** One origin, one certificate, one cache, one cookie domain. Cross-country comparison pages become possible, and v2's `userHasAccessToAllTenants: () => true` stops being a lie.
- **Cost.** No per-country domain and no per-country theme. A country cannot be handed to a partner as their own site.
- **Changes our mind.** A funder or partner contractually requiring their own branded domain.

A3. No per-country editorial scoping at launch

Roles exist, but any account can edit any country's data.

- **Gain.** Ships sooner, and it is genuinely enough for one in-house team.
- **Cost.** The first partner newsroom cannot be given an account safely.
- **Changes our mind.** A partner gets an account before Phase 2, which moves this into Phase 1. It is a `where` clause and a field guard, not a migration, which is why §10.1 puts it first in Phase 2 rather than last.

A4. The core is itself a plugin

- **Gain.** One extension API, used by core before anyone else, so no integration is a special case. Any adapter can be deleted without unpicking the domain.
- **Cost.** Indirection in places that would otherwise be direct, a slower first month, and a discipline the team has to keep.
- **Changes our mind.** We are certain we will only ever run one verification backend and one AI provider. The abstraction is not free and should not be bought on a maybe.

A5. Superdesk verifies, and Check is specified but not built

- **Gain.** Fact-checkers work where they already work, and Check's roughly 2,000 lines leave the codebase.
- **Cost.** Ten unknowns that live on someone else's instance (§10.3), and an integration nobody can fully test until the newsroom configures it.
- **Changes our mind.** The native review queue proves good enough during the Phase 2 pilot, making Superdesk optional rather than planned.
- Launch does not wait on this. Superdesk is Phase 3, after going live (§9.6).

B. Six questions that need an answer

#	Question	Who can answer	Blocks
B1	**Status taxonomy and the five groups.** `ClaimStatus.group` drives every scorecard and has no sensible default	Editorial lead	Launch
B2	**Category list.** 8 to 15, localized, authored from scratch	Editorial lead	Launch
B3	**Verdict vocabulary.** v2 inherited Meedan's `unverified / verified / false / misleading`. For a promise tracker the question is usually whether a promise is accurately attributed, not whether it is true	Editorial lead	Superdesk vocabularies
B4	**The ten Superdesk instance details** (§10.3, Q1 to Q10): API prefix, content profile, vocabularies, desk ids, service account, and whether `http_push` exists	Whoever administers the instance	Phase 3
B5	**Map tiles and boundary data,** and their licences for commercial and embedded use	Tech lead	Maps in Phase 2
B6	**Rewrite or refactor,** deliberately left out of the spec. With no data to migrate this is now almost purely a code-reuse question (§2.7)	Tech lead	Phase 1 start

Answer B1 first. It is small, it is editorial, and scorecards come out wrong rather than empty if it is inconsistent.

C. Already decided, and deliberately out of scope

Arguing with these is cheap now and expensive later, so argue now. They were chosen, not overlooked, and §1.6 gives the reasoning.

Out of scope for v3: custom domains per country, per-country theming, public user accounts with bookmarking or alerts, real-time collaborative editing, automated promise detection from news, crowdsourced verification where citizens decide rather than submit evidence, and Elasticsearch or Algolia.

Each one is left open in the data model, so it can be added later without a migration.

D. What useful feedback looks like

"Looks good" tells us nothing. These questions do.

- Where does this make your job harder than it is today?
- What have we assumed about your workflow that is not true?
- What would you end up doing by hand that you expected the system to do?
- Which target in §1.7 is wrong, and what is the real number?
- What breaks on election week, when half a year's traffic arrives in three days?

Comment inline wherever you are reading this, or reply in the thread. A sentence on one real problem is worth more than a pass over all eleven documents.

Overview

Status: draft for review. Version 3.0.0-draft.3, last updated 2026-11-09.

PromiseTracker tracks what politicians and public institutions promise, checks whether those promises are kept, and publishes the record. v3 is a re-architecture around two ideas.

1. **Everything is a plugin, including the core.** A thin kernel defines the interfaces, a core plugin supplies the domain, and every external system is a replaceable adapter.
2. **One site, many countries.** No multi-tenancy. A country is a path segment, as in `promisetracker.org/ke/politicians/william-ruto`, not a deployment boundary.

Why v3

v2 runs, but not without two third-party services.

- **Airtable** holds countries, politicians and source documents, and doubles as the operator console. Six pipeline tasks write human-readable status strings back into Airtable cells. `Processed = false` in an Airtable view is the work queue. No admin screen can create a document.
- **Meedan Check** holds published promises. Nothing in the codebase creates a promise record except the Check sync, and `promises.meedanId` is required and unique.

Remove either one and the product stops.

v2 has also never gone live. The code runs, but there is no production data and no public audience, so v3 inherits no legacy: no data migration, no URL history, no users to carry through a transition. That is the biggest freedom this re-architecture has, and the reason §9 is a launch plan rather than a migration plan.

Past the coupling, v2 carries structural problems that v3 fixes rather than inherits.

Problem	Evidence in v2
No promise history, though tracking over time is the whole pitch	`PromiseTimeline` is passed `events={[]}` and renders one synthetic point from `updatedAt`
The public site cannot scale	`force-dynamic` on every route, no caching, `depth: 6` queries, `getAllTenants()` re-run many times per render, and a promise list that loads every published promise unpaginated then filters in the browser
Search is decorative	`<Search />` renders with no handler. No search route, no index
Publication state follows three inconsistent conventions	`tenants.publish` (bool), `political-entities.publish` (bool), `promises.publishStatus` (free text, compared against the literal `"published"`)
Multi-tenancy is cosmetic	`userHasAccessToAllTenants: () => true`. Only 3 of 14 collections carry a `tenant`, and `tenant.country` is primary key, display country and subdomain label at once
Nothing distinguishes a politician from a ministry or a county	One `political-entities` collection with free-text `region` and `position`
`category` is free text	The UI offers filtering by it anyway. No taxonomy collection exists
Accented text is destroyed	`cleanText` in `src/tasks/extractDocuments.ts` strips `[^\x00-\x7F]` on a bilingual en/fr product

How to read this

Read in order if you are new. Jump to §2, §3 and §6 if you are implementing. If you are reviewing for the team, start with the review brief, which names the five decisions worth arguing about and the six questions that need an answer.

#	Document	For
1	Product	Problem, users, jobs to be done, core flows, scope and non-goals
2	Architecture	Kernel, plugins, ports and adapters, the plugin contract
3	Data model	Every collection and global, field by field
4	Governance	Publication state, the visibility cascade, roles
5	Pipeline	Ingest, extract, AI, review, publish
6	Plugins	Superdesk, native review, AI providers, ingest, Tika
7	Public site	Routing, caching, search, SEO, embeds, public API
8	Security and operations	Auth, secrets, SSRF, observability, deployment
9	Launch	Seeding, retiring Airtable and Check, launch sequence
10	Roadmap	Phasing, scale targets, open questions

Decisions this specification rests on

#	Decision	Argued in
D1	Superdesk Core is the fact-check review backend. The newsroom already runs it	§6
D2	v3 tracks a family of claim types, not only campaign promises	§3

#	Decision	Argued in
D3	Plugin-first, ports and adapters, five pluggable seams	§2
D4	Zero-integration mode is fully usable, on Mongo and S3 and nothing else	§2
D5	Read-only locking enforces plugin ownership	§2
D6	For Meedan Check, specify the port and defer the adapter	§6
D7	Claims have a real, first-class update history	§3
D8	AI extraction stays, radically simplified	§5
D9	Citizen input is native and moderated	§5
D10	No multi-tenancy. Country is a path segment and a field	§7
D11	The public site must fix caching, pagination, search, SEO and distribution	§7
D12	Rewrite versus refactor is deliberately out of scope here	§9
D13	The core is itself a plugin, and other plugins depend on and extend it	§2
D14	One `actors` collection with a `kind`: person, party, institution or place. Any actor can make promises	§3
D15	Typed URL segments: `/ke/politicians/<slug>`, `/ke/places/nairobi`	§7
D16	No per-country editorial scoping in v3, roles only	§4
D17	Places and politicians carry geodata, and the public site renders maps	§3
D18	Many sources, many formats. Ingest splits into connectors (where content comes from), content extractors (bytes to text) and structured importers (rows to records)	§5

Conventions

- **CONFIRM** marks a statement to verify against a running system before implementation. §10 collects all of them.
- File paths such as `src/utils/ssrf.ts` point at the v2 repository, as evidence or as code to reuse. They are not v3 paths.
- "Core" means `plugin-core`, the plugin that ships the domain. See §2.

---

1. Product

1.1 The problem

Politicians make promises to win support, and nobody revisits them. Information is not the gap. Manifestos, budget speeches and policy statements are all public. The gap is that nobody keeps the record in one place, over time, with evidence attached.

Journalists and civil-society watchdogs do this work today in spreadsheets and documents that nobody outside the newsroom sees, that fall out of date within months, and that cannot be queried, embedded or compared across countries.

PromiseTracker makes that record durable, public and defensible.

1.2 What v3 is

One public site covering many countries. Anyone can look up a politician, a party, a ministry or a county and see:

- every promise attributed to them, with the exact quote and a link to the source document
- the current status of each promise, and how that status changed over time, with evidence
- who verified it, when, and on what basis

Behind it sits an editorial system where a small team ingests source material from many places and formats, uses AI to find candidate promises, verifies them in whichever tool the newsroom already uses, and publishes. No spreadsheet anywhere in the loop.

1.3 Who it is for

Public audiences

User	What they need	What v3 gives them
Citizen	What did the person I voted for promise, and did they do it?	A politician profile with a scorecard, promise list and per-promise timeline
Journalist	A checkable fact, fast, with provenance	Promise pages with source quotes, document links and the verification record, plus an embeddable widget and a read-only JSON API
Researcher or CSO	Comparable data across politicians, regions and time	Filterable listings, bulk export, the public API
Partner newsroom	To put the tracker in front of their own audience	Embeddable scorecards and promise cards

Internal users

Role	Job
Contributor	Adds source documents and actors, prepares records, cannot publish
Reviewer or fact-checker	Triages AI-extracted candidates, verifies claims, records verdicts
Editor	Decides what goes public, owns promise updates and the timeline
Moderator	Triages citizen submissions
Administrator	Manages users, integrations, credentials and taxonomies

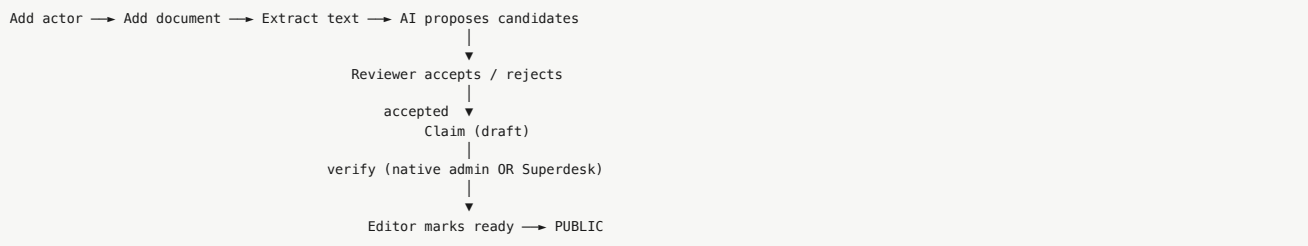
1.4 Jobs to be done

The system's obligations, in priority order.

1. **Get source material in, whatever shape it arrives in.** A manifesto PDF, a Word policy paper, a budget spreadsheet, a CSV of promises a partner already compiled, a URL to a speech transcript. (§5)
2. **Turn a document into candidate promises without reading all 200 pages.** AI proposes, a human disposes, and nothing reaches the public unreviewed. (§5)
3. **Verify a claim in the tool the newsroom already works in.** Superdesk for this newsroom, the Payload admin for a team that has nothing. (§6)
4. **Record what happened, when, and how we know.** Not a status field. A dated history with narrative and evidence. This is the product. (§3)
5. **Publish safely.** One switch controls visibility, and hiding a document immediately hides everything underneath it. (§4)
6. **Stay fast and findable under load,** on election day, linked from a national broadcaster. (§7)
7. **Let the work travel,** through embeds, a public API, bulk export, and structured data for search engines and AI assistants. (§7)
8. **Add a country without writing code.** (§7)

1.5 Core flows

A. Publishing a promise



Every step is visible in the admin. A document's position in the pipeline is one field, not something inferred from which relations happen to be populated.

B. Updating a promise over time

This is what makes it a tracker. An editor adds a claim update: a date, the status as of that date, a short narrative, and one or more pieces of evidence. The promise's current status comes from the most recent published update and is never typed in directly. The public page renders the full history as a timeline.

A citizen submission that survives moderation becomes a claim update, with attribution.

C. Citizen submission



Submissions are never published directly and are never visible to the public.

D. Adding a country

An administrator creates a country record with an ISO code and a name. It immediately has a landing page at `/{code}`, listings for each actor type, and a sitemap. No deployment, no configuration, no new Airtable base.

1.6 Scope

In scope for v3

- Countries, actors (politicians, parties, institutions, places), claims, claim updates, documents, categories, statuses, citizen submissions, media
- Multi-format, multi-source ingest, and AI-assisted extraction with human review
- A pluggable verification backend, with Superdesk and a native in-admin reviewer
- The public site: country and actor pages, filterable claim listings, claim pages with timeline and map, search
- Embeddable widgets and a read-only public JSON API
- Bulk CSV export
- English and French

Explicit non-goals for v3

Named so nobody has to guess, and so the data model can take them later.

Non-goal	Why	Left open how
Per-country editorial permissions	D16. One in-house team for now	Roles exist, and a `countries[]` filter can be added without migration (§4)
Custom domains per country	Not needed. Path segments cover it	Routing resolves country from the path, so a domain-to-country map is additive
Per-country theming	One brand today	Branding fields live on the country record
Public user accounts	No bookmarking, alerts or profiles	Submissions are anonymous with optional contact
Real-time collaborative editing	Payload's document locking is enough	
Automated promise tracking from news	Needs a reliable corpus and far more verification	The connector port would host it (§5)
Crowdsourced verification	Verification stays with accountable fact-checkers	Submissions carry evidence, and a human decides
Elasticsearch or Algolia	MongoDB text search is adequate at the target volumes	One module isolates the search layer (§7)

1.7 How success is measured

Dimension	Measure	Target
Editorial throughput	Median time from document upload to first published promise	< 1 working day

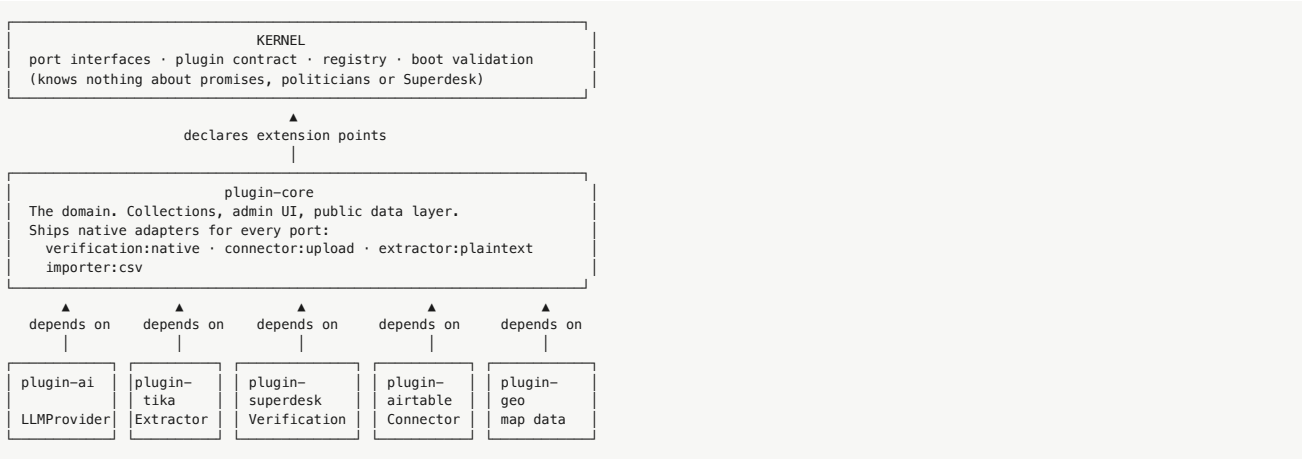
Dimension	Measure	Target
Review burden	Share of AI candidates accepted without edits	> 50%
Freshness	Share of published promises with an update in the last 6 months	> 70%
Reach	Embeds served, API requests, referrals from search	Trending up
Onboarding	Time to stand up a new country	< 1 hour, by a non-developer
Resilience	p95 page load under 10x normal traffic	< 2.5s LCP
Independence	External services required to run the system	0

The last row matters most. It is the measure v2 fails.

---

2. Architecture

2.1 Shape of the system



Three layers with strict boundaries.

Kernel. Interfaces, the plugin contract, a registry, and startup validation. It holds no domain concepts and could host a different product.

plugin-core (D13). Every domain collection, the admin experience, the public data layer, and a working implementation of every port. Enabling only core gives a complete, publishable PromiseTracker whose sole external dependencies are MongoDB and S3 (D4).

Integration plugins. Each declares `dependsOn: ["core"]`, extends core's collections through published extension points, and registers one or more port implementations.

Why the core is a plugin (D13)

If core were the app and plugins were decoration, every integration would reach into app internals and the boundary would rot. That is how v2 ended up with Airtable string-writing scattered across six pipeline tasks. Making core a plugin forces it to publish the same extension API everyone else uses. If an extension point is not good enough for `plugin-superdesk`, core feels it too.

It also makes core replaceable in principle. A deployment with a different domain model swaps `plugin-core` and keeps the kernel, the pipeline shape and the integrations.

2.2 Ports

A port is a TypeScript interface in the kernel. An adapter implements it and registers under a plugin-qualified id.

Port	Cardinality	Question it answers	Core adapter	Other adapters
`VerificationProvider`	exactly one	Where do humans verify claims?	`native` (review in the admin)	`superdesk`, and `check` (specified, not built)
`SourceConnector`	zero or more, `upload` always on	Where does content come from?	`upload`, `url`	`airtable`, `sheets`, `feed`
`ContentExtractor`	one or more, chosen by MIME type	How do bytes become text?	`plaintext` (txt, md, html)	`tika` (PDF, DOCX, XLSX and more), `ocr`
`StructuredImporter`	zero or more	How do tabular rows become records?	`csv`, `xlsx`	`sheets`
`LLMProvider`	many registered, one selected	Which model extracts candidates?	none, since AI is optional	`google`, `openai`, `anthropic` and others

Splitting ingest into three ports is what makes D18 work. Ingesting a CSV of promises a partner compiled and ingesting a 200-page manifesto PDF are different problems. One is a row-to-record mapping that needs no AI. The other is text extraction followed by AI extraction. v2 had one path and assumed PDFs.

Interfaces

```
// kernel/ports.ts

export interface VerificationProvider {
  readonly id: string
  /** Send a claim for human verification. Returns the external reference, if any. */
  submit(claim: ClaimRef, ctx: PortContext): Promise<ExternalRef | null>
  /** Pull verdicts changed since a cursor. Used for reconciliation and backfill. */
  pull(since: SyncCursor, ctx: PortContext): Promise<VerdictUpdate[]>
  /** Handle an inbound push from the provider, if it supports one. */
  receive?(payload: unknown, headers: Headers, ctx: PortContext): Promise<VerdictUpdate[]>
  health(): Promise<HealthResult>
}

export interface SourceConnector {
  readonly id: string
  readonly mode: 'push' | 'pull' | 'manual'
  /** Pull mode only: list items awaiting ingest. */
  list?(since: SyncCursor, ctx: PortContext): Promise<SourceItem[]>
  fetch(item: SourceItem, ctx: PortContext): Promise<FetchedContent>
  health(): Promise<HealthResult>
}

export interface ContentExtractor {
  readonly id: string
  /** MIME types this extractor claims, most specific wins; '*' is the fallback. */
  readonly accepts: readonly string[]
  extract(input: FetchedContent, ctx: PortContext): Promise<ExtractedText>
  health(): Promise<HealthResult>
}

export interface StructuredImporter {
  readonly id: string
  readonly accepts: readonly string[]
  /** Inspect without writing: used to build and preview a column mapping. */
  inspect(input: FetchedContent): Promise<TableShape>
  import(input: FetchedContent, mapping: ImportMapping, ctx: PortContext): Promise<ImportResult>
}

export interface LLMProvider {
  readonly id: string
  readonly models: readonly ModelDescriptor[]
  generate(req: GenerateRequest, ctx: PortContext): Promise<GenerateResult>
  health(): Promise<HealthResult>
}
```

`PortContext` carries the Payload instance, a scoped logger, an `AbortSignal`, and the resolved settings group for that plugin. Adapters never read `process.env` directly and never import from another plugin.

2.3 The plugin contract

A plugin is a factory returning a manifest. The kernel does the wiring.

```
export interface PluginManifest {
  readonly id: string                  // 'core' | 'superdesk' | 'ai' | ...
  readonly version: string
  readonly dependsOn?: readonly string[]

  /** Collections and globals this plugin owns outright. */
  collections?: CollectionConfig[]
  globals?: GlobalConfig[]

  /** Changes to collections owned by another plugin. */
  extend?: Record<string, CollectionExtension>

  /** Port implementations, keyed by port name. */
  provides?: {
    verification?: VerificationProvider
    connectors?: SourceConnector[]
    extractors?: ContentExtractor[]
    importers?: StructuredImporter[]
    llm?: LLMProvider[]
  }

  tasks?: TaskConfig[]
  workflows?: WorkflowConfig[]
  /** Declared, not registered: App Router routes are filesystem entries. */
  routes?: PluginRoute[]
  /** A group appended to the Settings global, namespaced by plugin id. */
  settings?: Field[]
  /** Validated at boot when the plugin is enabled. */
  requiredEnv?: readonly string[]
}

export interface CollectionExtension {
  /** Appended fields. Always optional, see §2.5. */
  addFields?: Field[]
  /** Core field names this plugin becomes the authority for. */
  ownFields?: string[]
  /** Hide create and delete: this plugin owns the record lifecycle. */
  ownLifecycle?: boolean
  /** A banner shown on the edit view explaining who owns this data. */
  notice?: Record<string, string>
  hooks?: Partial<CollectionHooks>
}
```

One environment variable lists the enabled plugins, and port selection is explicit.

```
PT_PLUGINS=core,tika,ai,superdesk       # core is implied but may be listed
PT_VERIFICATION=superdesk               # which VerificationProvider wins
PT_LLM_MODEL=google:gemini-2.5-pro      # which LLMProvider and model
```

Plugin choice is evaluated at boot, not at runtime. Field injection and route registration cannot be toggled per request, and a mid-session change of ownership would create exactly the conflicts §2.5 exists to prevent.

Boot sequence

1. Read `PT_PLUGINS` and load each manifest.
2. Verify `dependsOn` is satisfied, then sort topologically. A cycle is a fatal error.
3. Resolve the selected adapter for each exclusive port. Zero or more than one is a fatal error, and the message names the candidates and the variable that picks between them.
4. Merge collections, then apply `extend` in dependency order.
5. Append each plugin's settings group to the Settings global under its id.
6. Validate `requiredEnv` for enabled plugins only.
7. Register tasks and workflows.
8. Log the resolved configuration as one structured line: plugins, port bindings, adapter versions. This is the first thing anyone will want when debugging a deployment.

Failing fast at boot departs from the v2 reference architecture, where disabled plugin routes returned `503` at request time. That suits a genuinely optional endpoint. It is the wrong behaviour for a required capability with interchangeable implementations. A deployment with no verification provider is misconfigured and should not start.

2.4 Mode matrix

Enabled	Ingest	Text extraction	Candidate generation	Verification
`core`	Admin upload, URL, CSV and XLSX import	Plain text and HTML only	None. Promises entered by hand	Review queue in the admin
`core,tika`	as above	PDF, DOCX, XLSX, PPTX, ODF, RTF and more	None	Admin
`core,tika,ai`	as above	as above	AI candidates into the review queue	Admin
`core,tika,ai,superdesk`	as above	as above	AI candidates into the review queue	Superdesk
`core,tika,ai,superdesk,airtable`	plus Airtable	as above	as above	Superdesk

The first row keeps everyone honest. It is a complete product, it is what CI runs against, and it is what a new contributor gets from `pnpm dev` with no credentials.

2.5 Ownership and single source of truth (D5)

When a plugin owns data, exactly one writer exists. The kernel and code review enforce six rules.

1. **Injected fields are always optional** and carry a sensible default. A plugin may be removed after months of use, and required fields would invalidate every existing record.
2. **A plugin may not change a core field's type,** only its admin presentation and access.
3. **Owned fields become read-only in the admin** and reject writes from anyone but the owning plugin and a super-admin, whose override is audit-logged.
4. **Lifecycle ownership hides create and delete** for that collection and shows a notice explaining where the data comes from.
5. **Plugin fields are namespaced** under a group named for the plugin, so `superdesk.guid` rather than a bare `guid`. Two plugins can never collide, and removal is a clean sweep.
6. **A plugin never deletes a core record.** Upstream deletion maps to `recordStatus: archived` (§4). Deleting a record with downstream work, such as extracted claims, completed verification or a published timeline, would destroy irreversible effort silently.

Example: `plugin-superdesk` ***taking ownership of verification***

```
extend: {
  claims: {
    addFields: [
      {
        name: 'superdesk',
        type: 'group',
        admin: { readOnly: true, condition: () => true },
        fields: [
          { name: 'guid', type: 'text', index: true },
          { name: 'version', type: 'number' },
          { name: 'state', type: 'text' },
          { name: 'itemUrl', type: 'text' },
          { name: 'lastSyncedAt', type: 'date' },
          { name: 'lastSyncStatus', type: 'select', options: ['success', 'failed', 'pending'] },
          { name: 'lastSyncError', type: 'text' },
        ],
      },
    ],
    ownFields: ['verification.verdict', 'verification.verifiedAt', 'verification.rationale'],
    notice: {
      en: 'Verification for this claim is managed in Superdesk. Changes made here will be overwritten on the next sync.',
      fr: 'La vérification de cette affirmation est gérée dans Superdesk. Les modifications effectuées ici seront écrasées lors de la prochaine
synchronisation.',
    },
  },
}
```

Note what it does not own: title, statement, category, promisor, the update timeline. Those stay editable in PromiseTracker whichever verification backend is active. Only the verdict crosses the boundary.

2.6 Technology

Layer	Choice	Why
Framework	Next.js (App Router), React 19, TypeScript strict	Carried from v2. The admin and the public site share one deployment
CMS	Payload CMS 3.x	Admin UI, a plugin API strong enough for §2.3, typed, a job queue, Next-native
Database	MongoDB	Flexible schema for an evolving model, Payload's best-supported adapter, already in production
Media	S3-compatible	Already in production
Text extraction	Apache Tika, self-hosted	Open source, no per-call cost, over 1,000 formats
AI	Vercel AI SDK behind `LLMProvider`	Already in v2 as `src/lib/ai/providerRegistry.ts`, and becomes the adapter registry
Errors	Sentry	Already instrumented in `src/tasks/utils.ts` and `src/workflows/utils.ts`
Jobs	Payload Jobs	Already in use, and workflow locking exists in `src/lib/workflowLocks.ts`
Tests	Vitest for integration, Playwright for e2e	Already in use, and the security specs are the regression floor

No new infrastructure. The change is structural, not operational.

2.7 Reused from v2

Carried forward deliberately, not rewritten.

What	Where in v2	Why i
Download hardening	`src/utils/ssrf.ts`, `src/utils/files.ts`, `src/utils/fileSignature.ts`	HTTPS resolut hop, pr rejectio magic- Docum `docs/ file- inges` Hard t already
Quote verification	`src/tasks/extractPromises.ts`, functions `hasVerifiableQuote` and `mergeByAtomicPromise` at lines 189 to 213	Drops whose literall source. halluci and it i
AI provider registry	`src/lib/ai/providerCatalog.ts`, `providerRegistry.ts`, around 800 lines	Already but nar the `LL` adapte
Route grammar	`src/lib/routes/publicRoutes.ts`	Pure re with th resolve never r

What	Where in v2	Why i
Encrypted settings	`src/fields/encryptedSecret.ts`	Encryp masks field ac admins the valu mask is
Slug and link fields	`src/fields/slug/*`, `src/fields/link/*`	Fine as
Workflow locking	`src/lib/workflowLocks.ts`	Atomic `findO` cron gu
Media deduplication	`src/collections/Media.ts` and `downloadDocuments`	Source `exter` SHA-2
Observability wrappers	`src/tasks/utils.ts`, `src/workflows/utils.ts`	Sentry structu workfl
Security tests	`tests/int/{ssrf,fileSignature,requestGuards,securityHeaders,privilegedAccess,publicApiAccess,publicRoutes}.int.spec.ts`	These rules. T floor, n point

One bug to fix, not port

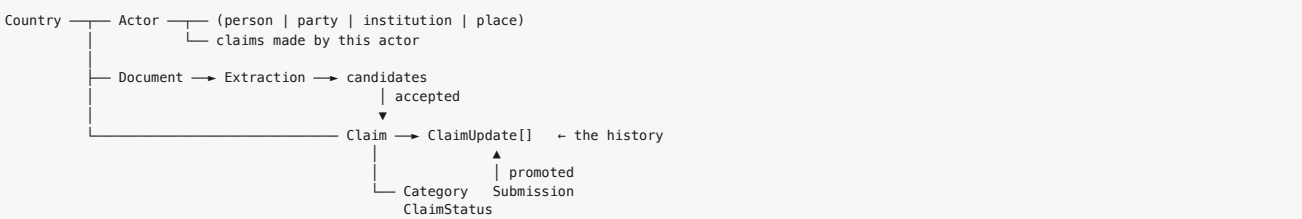
`cleanText` in `src/tasks/extractDocuments.ts:19` strips every non-ASCII character with `.replace(/[^\x00-\x7F]/g, "")`. On a bilingual English and French product this silently corrupts every accented word in every French document, and it would do the same to Portuguese, Swahili orthography and any African language with diacritics. v3 normalises Unicode to NFC and collapses whitespace. It discards no characters.

---

3. Data model

`plugin-core` owns everything here unless stated otherwise. Payload manages `createdAt` and `updatedAt` on every collection, so they are not listed. §4 explains `recordStatus` and the computed `_status`. Read it before implementing any hook here.

3.0 At a glance



Against v2: `tenants` is gone (D10), `political-entities` becomes `actors` with a kind (D14), `promises` becomes `claims` with a type (D2), `claim-updates` is new and is the point of the product (D7), `promise-status` is authored locally instead of synced from Meedan, `category` becomes a real taxonomy, and `global-pages`, `global-site-settings`, `entity-page`, `promise-updates` and `ai-extraction-export-rows` all disappear.

3.1 Countries

Replaces v2's `tenants`. A country is a routing namespace and a scoping field, not a tenant. One deployment, one database, one set of users (D10).

```
interface Country {
  id: string
  code: string          // ISO 3166-1 alpha-2, lowercase. UNIQUE, INDEXED. The URL segment: "ke"
  alpha3?: string       // ISO 3166-1 alpha-3, for data interchange
  name: string          // localized
  officialName?: string // localized
  flag?: Media
  defaultLocale: 'en' | 'fr'
  locales: ('en' | 'fr')[]       // which locales this country publishes in
  geo?: GeoRef                   // §3.9. Centroid and bounds for the country map
  about?: RichText               // localized, shown on the country landing page
  branding?: {
    accentColor?: string
    heroImage?: Media
  }
  partners?: Partner[]
  meta?: SeoMeta
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'
}
```

Indexes: `code` (unique), `recordStatus`.

`code` is both the identity and the URL segment, which is the overload D10 removes from v2's `tenant.country`. Per-country `locales` lets a Kenyan tracker be English-only while a Senegalese one is French-only. Archiving a country hides every actor, claim and document under it (§4).

3.2 Actors

One collection, four kinds (D14). A politician, a party, a ministry and a county all make promises. Modelling them separately would mean three more collections, three more route trees, and a union query every time you ask who makes promises here.

```
type ActorKind = 'person' | 'party' | 'institution' | 'place'

interface Actor {
  id: string
  country: Country              // required, INDEXED
  kind: ActorKind               // required, INDEXED
  name: string                  // required, useAsTitle
  shortName?: string            // "UDA", "MoH". Used in dense UI
  slug: string                  // unique per (country, kind). INDEXED
  parent?: Actor                // ward → constituency → county; department → ministry
  image?: Media
  description?: RichText        // localized
  links?: { label: string; url: string }[]   // official site, social
  geo?: GeoRef                  // §3.9
  meta?: SeoMeta
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'

  // ── kind-specific, shown conditionally in the admin ──

  person?: {
    honorific?: string
    gender?: string
    dateOfBirth?: string
    originPlace?: Actor         // kind='place'. Where they are from, drives the map
    offices: Office[]           // at least one
  }

  party?: {
    abbreviation?: string
    founded?: string
    colors?: { primary?: string; secondary?: string }
    leader?: Actor              // kind='person'
  }

  institution?: {
    institutionType: 'ministry' | 'agency' | 'county-government' | 'parastatal'
                   | 'commission' | 'legislature' | 'other'
    jurisdiction?: Actor        // kind='place'
    headedBy?: Actor            // kind='person'
    establishedOn?: string
  }

  place?: {
    placeType: 'region' | 'province' | 'state' | 'county' | 'district'
             | 'constituency' | 'ward' | 'city' | 'other'
    adminLevel: number          // 1 = first-level subdivision, 2 = second, and so on
    population?: number
    externalCode?: string       // GADM / geoBoundaries / HASC identifier
  }
}

interface Office {
  title: string                 // "President", "Governor", "Cabinet Secretary for Health"
  institution?: Actor           // kind='institution'
  jurisdiction?: Actor          // kind='place'. "Governor of Nairobi"
  party?: Actor                 // kind='party'. Party at the time of this office
  from: string                  // required
  to?: string                   // open-ended while in office
  isCurrent: boolean            // derived from `to`, stored for cheap querying, INDEXED
}
```

Indexes: `(country, kind, slug)` unique, `(country, kind, recordStatus)` for listings, `parent`, `person.offices.isCurrent`.

Slug uniqueness is scoped to `(country, kind)`. A politician and a county may both be slugged `nairobi` without colliding, because typed URL segments keep them apart (D15). v2's `ensureUniqueSlug` enforced per-tenant uniqueness, and the same hook gains the kind dimension.

`offices[]` replaces v2's flat `position`, `region`, `periodFrom` and `periodTo`, which could represent only one office and lost a politician's history the moment they changed job.

`originPlace` powers "where this politician is from" on the map. `offices[].jurisdiction` powers "where they govern". The two are often different, and v2 could express neither.

3.3 Claims

Replaces v2's `promises`. Typed (D2), so one collection, one pipeline and one review queue serve campaign promises and other verifiable claims.

```
type ClaimType = 'campaign-promise' | 'policy-pledge' | 'budget-commitment' | 'public-statement'

interface Claim {
  id: string
  country: Country              // required, INDEXED. Denormalised from promisor for query speed
  type: ClaimType               // required, INDEXED. default 'campaign-promise'
  title: string                 // required, localized. The editorial headline
  slug: string                  // unique per country. INDEXED
  statement: string             // required. The exact words, verbatim, never paraphrased
  summary?: RichText            // localized. Editorial context

  promisor: Actor               // required, INDEXED. Who made it
  coPromisors?: Actor[]         // for example the party as well as the candidate
  jurisdiction?: Actor          // kind='place'. Where it applies, if narrower than the country

  category: Category            // required, INDEXED
  tags?: string[]

  source?: Document             // INDEXED. May be absent for a manually entered claim
  sourceLocator?: {
    page?: number
    chunk?: number
    timestamp?: string          // for audio and video sources
    excerptStart?: number
  }
  sourceUrl?: string            // attribution only. NEVER fetched by the backend (§8)
  madeOn?: string               // when the promise was made
  targetDate?: string           // the deadline it set itself, when it set one
  language: 'en' | 'fr'

  // ── tracking ──
  currentStatus?: ClaimStatus   // READ-ONLY. Derived from the latest published ClaimUpdate
  statusAsOf?: string           // READ-ONLY. Date of that update
  updateCount?: number          // READ-ONLY. Denormalised for listings

  // ── verification ──
  verification: {
    verdict: 'unverified' | 'verified' | 'false' | 'misleading'   // default 'unverified'
    verifiedAt?: string
    verifiedBy?: User | string  // a user, or an external reviewer's name
    rationale?: RichText
    report?: Media              // a published fact-check artefact
  }

  // ── provenance ──
  extraction?: Extraction       // set when AI-derived
  confidence?: number           // 0 to 1, from the extraction
  enteredBy: 'ai' | 'manual' | 'import'   // INDEXED. Lets you audit AI quality

  image?: Media
  meta?: SeoMeta
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold', see §4.2
}
```

Indexes: `(country, recordStatus, _status)`, `(promisor, _status)`, `(country, category, _status)`, `(country, currentStatus)`, `slug` unique per country, `verification.verdict`, and a text index on `title`, `statement` and `summary` (§7.3).

Versions and drafts are enabled. Every published change is recoverable, and an editor can stage a correction without taking the page down.

No human ever writes `currentStatus`. An `afterChange` hook on `claim-updates` recomputes it from the claim's updates. This is the structural fix for v2, where status was a field someone set and the timeline was fiction.

`statement` is the verbatim quote, and it is what the AI quote-verification guard checks against the source document. A candidate whose quote cannot be found in the source text is dropped (§5.5).

`enteredBy` makes "how good is our extraction, really?" answerable, which the v2 model could not do once a promise round-tripped through Check.

`verification.verdict` is the only field a verification plugin owns (§2.5).

3.4 Claim statuses

Authored in the CMS. In v2 this collection was seeded from Meedan's `verification_statuses`, joined on `meedanId` in three places, and only ever created, never updated, so a label fixed in Check never reached the site.

```
interface ClaimStatus {
  id: string
  slug: string                  // unique, stable: 'kept', 'in-progress', 'broken', and so on
  label: string                 // required, localized
  description: string           // required, localized. The public "what does this mean?" copy
  group: 'not-started' | 'in-progress' | 'kept' | 'broken' | 'inconclusive'   // required
  color: string                 // hex, used for chips and charts
  textColor: string             // hex, default '#ffffff'
  order: number                 // display order
  isDefault?: boolean           // the status a new claim's first update suggests
}
```

`group` is new and it earns its place. v2's profile charts had to be configured per page by picking statuses into three `chartGroups` on a `hero` block, with a hardcoded fallback and a balancing redistribution when the configuration was absent. Grouping belongs on the status, not on a page block. Define it once and every chart, filter and scorecard derives from it.

Statuses are global rather than per country, which makes cross-country comparison meaningful.

3.5 Claim updates

New in v3, and the reason v3 exists (D7).

```
interface ClaimUpdate {
  id: string
  claim: Claim                  // required, INDEXED
  date: string                  // required, INDEXED. When the development happened,
                                // not when it was recorded
  status: ClaimStatus           // required. The status as of this date
  title?: string                // localized. A short headline for the timeline entry
  narrative: RichText           // required, localized. What happened, and how we know
  evidence: Evidence[]          // at least one for a status change
  author?: User
  attribution?: string          // credit for a citizen contributor, with consent
  sourceType: 'editorial' | 'citizen-submission' | 'import' | 'automated'
  submission?: Submission       // set when promoted from the moderation queue
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'
}

interface Evidence {
  label: string                 // localized
  kind: 'link' | 'document' | 'media' | 'quote'
  url?: string                  // attribution only, never fetched
  document?: Document
  media?: Media
  quote?: string
  publisher?: string
  publishedOn?: string
}
```

Indexes: `(claim, date)` descending for the timeline query, and `(claim, recordStatus)`.

On create, update or delete of a published `ClaimUpdate`, a hook sets three fields on the parent claim.

```
claim.currentStatus = status of the most recent ClaimUpdate where _status === 'published'
                      ordered by date desc, then createdAt desc
claim.statusAsOf     = that update's date
claim.updateCount    = count of published updates
```

With no published updates, `currentStatus` is null and the public page shows the claim as not yet assessed. A claim with a verdict but no updates is a promise recorded but not yet tracked, which is a legitimate and visible state.

`date` is separate from `createdAt` because promises are tracked retrospectively. An editor in 2026 records that a hospital opened in 2024. The timeline must order by when things happened, not by when someone got round to typing them.

3.6 Categories

A real taxonomy, replacing v2's free-text `promises.category`.

```
interface Category {
  id: string
  slug: string                  // unique, INDEXED
  name: string                  // required, localized
  description?: string          // localized
  icon?: Media
  color?: string
  parent?: Category             // one level of nesting, no deeper
  order: number
}
```

Global rather than per country, which makes "healthcare promises across Africa" a query rather than a reconciliation exercise.

3.7 Documents

Source material. The headline change is one explicit processing state, replacing v2's state scattered across `files exists`, `extractedText.text exists`, `fullyProcessed`, `extractionReview` and a status string written into an Airtable cell.

```
type ProcessingStatus =
  | 'pending' | 'fetching' | 'fetched'
  | 'extracting' | 'extracted'
  | 'analysing' | 'analysed'
  | 'imported'          // structured-import path, §5.6
  | 'completed' | 'failed' | 'needs-review'

interface Document {
  id: string
  country: Country              // required, INDEXED
  title: string                 // required
  actors?: Actor[]              // who this document is by or about
  type: 'manifesto' | 'speech' | 'budget' | 'policy' | 'report'
      | 'news' | 'dataset' | 'other'
  language: 'en' | 'fr'
  publishedOn?: string
  publisher?: string

  file?: Media                  // the stored artefact
  sourceUrl?: string            // attribution. Fetched ONLY via the `url` connector (§5.2)
  connector: string             // which SourceConnector brought it in: 'upload', 'url', ...
  mimeType?: string             // detected, not trusted from the client

  processing: {
    status: ProcessingStatus    // required, INDEXED. default 'pending'
    stage?: string              // human-readable label for the current operation
    message?: string            // the last error or notice, shown in the admin
    extractorId?: string        // which ContentExtractor handled it
    startedAt?: string
    lastAttemptAt?: string
    completedAt?: string
    retryCount: number          // default 0. One budget across errors and stuck sweeps
  }

  text?: string                 // extracted plain text, Unicode-normalised (NFC)
  textChars?: number
  pageCount?: number

  recordStatus: 'ready' | 'hold' | 'archived'   // default 'ready'
}
```

Indexes: `(processing.status, processing.lastAttemptAt)` for the work queue and the stuck sweep, `(country, recordStatus)`, and `actors`.

`processing.status` is the work queue. v2's queue was `Processed = false` in an Airtable view, so removing Airtable removed the queue. Making it an indexed enum is the single highest-value change in the pipeline, and it decouples the pipeline from any ingest source.

`sourceUrl` is attribution only by default, exactly as the v2 reference architecture insists. The one path that fetches a user-supplied URL is the explicit `url` connector, which runs through the existing SSRF guard (§8).

Documents are not public. Claim pages link to them as evidence by URL, but the collection is admin-read-only, as in v2.

3.8 Extractions

The AI audit trail, and the review queue's backing store. Owned by `plugin-ai`.

```
interface Extraction {
  id: string
  document: Document            // required, INDEXED
  model: string                 // 'google:gemini-2.5-pro'
  promptVersion: string         // so a prompt change is attributable
  status: 'pending' | 'completed' | 'failed' | 'partial'
  error?: string
  tokensUsed?: number
  durationMs?: number
  rawResponse?: JSON            // retained for debugging, see retention in §8
  candidates: Candidate[]
}

interface Candidate {
  uniqueId: string              // stable, generated, survives re-extraction
  category?: string             // free text as proposed, mapped to a Category on acceptance
  summary: string               // the proposed claim title
  quotes: string[]              // verbatim spans, each verified against document.text
  confidence?: number
  decision: 'pending' | 'accepted' | 'rejected'   // INDEXED
  claim?: Claim                 // set on acceptance
  rejectedReason?: string
  reviewedBy?: User
  reviewedAt?: string
}
```

Re-extracting a document creates a new `Extraction` and leaves prior ones intact. Decisions already made survive, and extraction quality over time becomes measurable.

3.9 Geo (D17)

A shared field group, used by `Country` and `Actor`. Enough to draw a map without turning PromiseTracker into a GIS.

```
interface GeoRef {
  centroid?: { lat: number; lng: number }
  bounds?: { north: number; south: number; east: number; west: number }
  boundary?: Media              // a GeoJSON file, stored like any other upload
  boundarySource?: string       // 'geoBoundaries', 'GADM', 'OSM'. Attribution
  boundaryLicense?: string      // required when boundary is set
  externalCode?: string         // the id in that source, for refresh
  defaultZoom?: number
}
```

Boundary polygons are stored as uploaded GeoJSON rather than fetched live. Boundary datasets carry licence terms, change rarely, and a live third-party tile or geometry dependency is exactly the coupling v3 is removing. Attribution is required when a boundary is present.

The public map (§7) renders `boundary` when it exists and falls back to a marker at `centroid`. A country with neither gets no map, so the feature degrades rather than breaks.

Boundary files can be large. The admin warns above 2 MB and the public site serves a simplified rendition. **CONFIRM** the topology-preserving simplification step at implementation time.

3.10 Submissions

Citizen input (D9), replacing v2's embedded Airtable form whose rows nothing ever read back.

```
interface Submission {
  id: string
  claim?: Claim                 // INDEXED. Absent for "we're missing a promise" reports
  country?: Country
  kind: 'update' | 'correction' | 'new-claim'
  body: string                  // required
  evidenceUrl?: string          // attribution only. NEVER fetched
  attachment?: Media
  submitterName?: string
  submitterEmail?: string
  consentToAttribute: boolean   // default false
  status: 'new' | 'accepted' | 'rejected' | 'spam'   // INDEXED. default 'new'
  moderatorNote?: string
  moderatedBy?: User
  moderatedAt?: string
  claimUpdate?: ClaimUpdate     // set when promoted
  // request metadata, for abuse handling
  ipHash?: string               // salted hash, never the raw address
  userAgent?: string
}
```

Access: `create` is public and rate-limited, and `read` is authenticated only. A submission is never rendered publicly, in any status.

3.11 Supporting collections

Media. Unchanged from v2 in substance: `alt` (required), `externalUrl`, `checksum` (SHA-256, indexed, drives deduplication), S3-backed, public read.

Users. `roles` (`super-admin`, `editor`, `contributor`, `reviewer`, `moderator`, `viewer`, hasMany), `name`, and `countries?`, a relationship that exists but is not enforced in v3. See §4.6.

Pages. One collection, replacing v2's `pages` and `global-pages` split. An optional `country` relationship, where null means the page is global (`/about`) and set means it is a country page (`/ke/about`). Same block set either way, and blocks that need a country render empty when there is none.

Partners. `name`, `description`, `image`, `url`, optional `countries[]`.

WorkflowLocks. System collection, carried over verbatim from `src/collections/WorkflowLocks.ts`.

ImportJobs (`plugin-core`). One record per structured import: the file, the chosen `StructuredImporter`, the column mapping, a dry-run preview, counts of created, updated and skipped rows, and per-row errors. See §5.6.

3.12 Globals

Site settings. A single global, now that there is no tenancy: site title and description, logos, navigation, footer, legal links, social links, newsletter configuration, default SEO. Country-specific overrides live on the `Country` record rather than in a second global.

Settings. Integration credentials, super-admin read and write only. Core contributes the feature toggles, and each plugin appends a namespaced group (§2.3).

```
interface Settings {
  features: {
    aiExtractionEnabled: boolean
    submissionsEnabled: boolean
    publicApiEnabled: boolean
    embedsEnabled: boolean
  }
  // appended by plugins, namespaced by plugin id:
  superdesk?: { baseUrl; clientId; clientSecret; deskId; stageId; profileId; pushSecret }
  ai?: { credentials[]; model; limits }
  tika?: { url }
  airtable?: { apiKey; baseId }
}
```

Credentials live here rather than in environment variables because they change per deployment without a redeploy, and because `src/fields/encryptedSecret.ts` already encrypts them at rest and masks them on read. Environment variables select which plugins run. The database holds

what they need to run.

3.13 Localization

Content locales are `en` and `fr`, as in v2, driven by `NEXT_PUBLIC_LOCALES`.

Localized: `Country.name`, `officialName` and `about`, `Actor.name` and `description`, `Claim.title` and `summary`, `ClaimUpdate.title` and `narrative`, `Evidence.label`, `ClaimStatus.label` and `description`, `Category.name` and `description`, all page block text, and site settings text.

Not localized: slugs, `Claim.statement`, codes, dates, colours and numbers. A verbatim quote is not translated, since a translation would be a different object.

v2 shipped a French admin UI while leaving `promises.title`, `promises.description` and the political-entity fields untranslated. v3 localizes the content people actually read.

---

4. Governance

How a record becomes public, who may do what, and what happens when something is pulled down.

4.1 The problem this solves

v2 expresses publication three incompatible ways. `tenants.publish` is a boolean set by a sync task and read-only to editors. `political-entities.publish` is the same. `promises.publishStatus` is a free-text field compared against the string literal `"published"` in both the access rule (`src/collections/Promises.ts:12`) and the listing query. Nothing connects them. Putting a country on hold does not hide the politicians under it, and a typo in `publishStatus` silently unpublishes a promise.

v3 has one writable field and one computed one.

4.2 recordStatus and _status

`recordStatus` captures intent. It is the only publication field a human writes.

Value	Meaning
`ready`	This record is, on its own terms, fit to publish
`hold`	Not ready, or temporarily withdrawn. Default for claims and claim updates
`archived`	Retired. Preserved for audit, never public, never auto-deleted

`_status` captures the outcome. It is Payload's own draft and published state, computed and never set directly. The rule below is the whole of it.

```
record._status = 'published'  ⟺  record.recordStatus === 'ready'
                                 ∧ every ancestor's recordStatus === 'ready'
                                 ∧ (claims only) verification.verdict === 'verified'
               else 'draft'
```

Ancestry

Record	Ancestors that must all be `ready`	Extra condition
`Country`	none	none
`Actor`	country	none
`Document`	country	none
`Claim`	country, promisor, source document when set	`verification.verdict === 'verified'`
`ClaimUpdate`	country, promisor, claim	none
`Submission`	never published	none

A claim's source document counts as an ancestor because the document is the evidence. If a manifesto is withdrawn as inauthentic, every promise extracted from it has to vanish with it. In v2 nothing would have happened at all.

Why two fields

A single "published" checkbox cannot express "this promise is fine but the document it came from is under review". With two fields, intent survives the cascade. A promise an editor held individually stays held when its parent is released, and a promise that was hidden only because its parent was held comes back automatically. There is no blanket "set all descendants to ready", because that would silently override a human's decision.

4.3 The cascade

An `afterChange` hook fires when `recordStatus` changes on any collection.

1. Recompute `_status` for the changed record, synchronously. The save returns immediately with the record correct.
2. Enqueue a background job to walk descendants and recompute each one independently from its own `recordStatus` plus the refreshed ancestry.
3. On completion, call `revalidateTag` for every affected cache tag (§7.2).

Downward, when a parent goes to `hold` or `archived`, every descendant recomputes to `draft` whatever its own intent, and the subtree leaves the public site within seconds.

Upward, when a parent returns to `ready`, each descendant recomputes from its own `recordStatus`, so individually held records stay held.

A status change never triggers re-extraction, re-verification or any other pipeline work. Existing claims stay valid. Only their visibility moves.

Because `_status` is always accurate in the database, every public query stays trivial, with no ancestry joins and no `$lookup`.

```
await payload.find({
  collection: 'claims',
  where: { and: [{ _status: { equals: 'published' } }, { country: { equals: countryId } }] },
})
```

Scale note

A country with 5,000 claims and 20,000 updates cascades in one background job with batched bulk updates rather than one write per record. The job is idempotent, safe to re-run, and reports counts to Sentry. For very large subtrees it chunks and checkpoints, so a restart resumes rather than starts over.

4.4 Deletion

No sync, import or plugin ever hard-deletes a record. Upstream deletion maps to `recordStatus: archived`. Hard deletion is an explicit admin action, taken after review, on a record the admin has looked at.

The reason is concrete. Deleting a document that has extracted claims, human verification decisions and a published timeline destroys irreversible work, and upstream systems are unreliable about announcing deletions in the first place.

4.5 Roles

Roles are additive. A user with `contributor` and `reviewer` can do both.

Role	Purpose
`super-admin`	Full access. Manages users, settings, plugin credentials and taxonomies. The only role that can override a plugin-owned field, and that override is audit-logged
`editor`	Final content authority. Full `recordStatus` control on every collection. Owns the claim timeline
`contributor`	Adds actors, documents and draft claims. Cannot publish, and may set `recordStatus` to `hold` only
`reviewer`	Triages extraction candidates and sets `verification.verdict`. Disabled when a verification plugin owns that field
`moderator`	Works the submissions queue, and may promote an accepted submission to a claim update
`viewer`	Read-only

Permission matrix

Action	super-admin	editor	contributor	reviewer	moderator	viewer
Create or update countries	✓	✓				
Create or update actors	✓	✓	✓			
Create or update documents	✓	✓	✓			
Create claims manually	✓	✓	✓			
Accept or reject extraction candidates	✓	✓		✓		
Set `verification.verdict`	✓	✓		✓		
Create or update claim updates	✓	✓			✓ from a submission	
Set `recordStatus`	any value	any value	`hold` only	on claims, any		
Moderate submissions	✓	✓			✓	
Hard-delete	✓	✓				
Override a plugin-owned field	✓, logged					
Manage users and settings	✓					
Read all records	✓	✓	✓	✓	✓	✓

Plugin and role symmetry

When a plugin owns a capability, the matching in-CMS role loses it, so two sources of truth never exist.

Active plugin	Capability it takes	Effect
`plugin-superdesk`	`verification.verdict`	`reviewer` can no longer set a verdict in the admin, because Superdesk is the review authority. Candidate triage stays in PromiseTracker
`plugin-airtable`	Document and actor intake	`contributor` creation of those records is disabled, because Airtable is the entry point

The claim timeline is always editorial, whichever plugins are active.

4.6 What v3 does not do (D16)

There is no per-country editorial scoping. Any authenticated user with a write role can edit any country's records. `User.countries` exists in the schema as an unenforced relationship.

This is a deliberate choice for a single in-house team, and it is the right trade while that stays true. It stops being right the moment a partner newsroom gets an account, because nothing would prevent a Kenyan contributor from editing South African records, deliberately or by accident.

Turning it on later is a `where` clause in the collection access functions plus a field-level guard, with no migration. The relationship is already there, and country is already a required, indexed field on every content collection. §10 tracks it as the first thing to add when the user base widens.

v2's position was worse but less honest. It ran `@payloadcms/plugin-multi-tenant` with `userHasAccessToAllTenants: () => true`, which looks like isolation and provides none.

4.7 Audit

Event	Recorded where
Any `recordStatus` change	Payload version history, plus a structured log with actor, record, old and new value
`verification.verdict` change	Version history, plus `verifiedBy` and `verifiedAt` on the record
Candidate accept or reject	`Candidate.reviewedBy`, `reviewedAt`, `rejectedReason`
Submission moderation	`moderatedBy`, `moderatedAt`, `moderatorNote`
Super-admin override of a plugin-owned field	Structured log at `warn` plus a Sentry breadcrumb, with before and after
Plugin sync writes	The plugin's own `lastSyncedAt`, `lastSyncStatus` and `lastSyncError` group

Claims and claim updates have versions enabled, so "what did this page say in March?" is answerable without a separate audit store.

4.8 Admin notifications

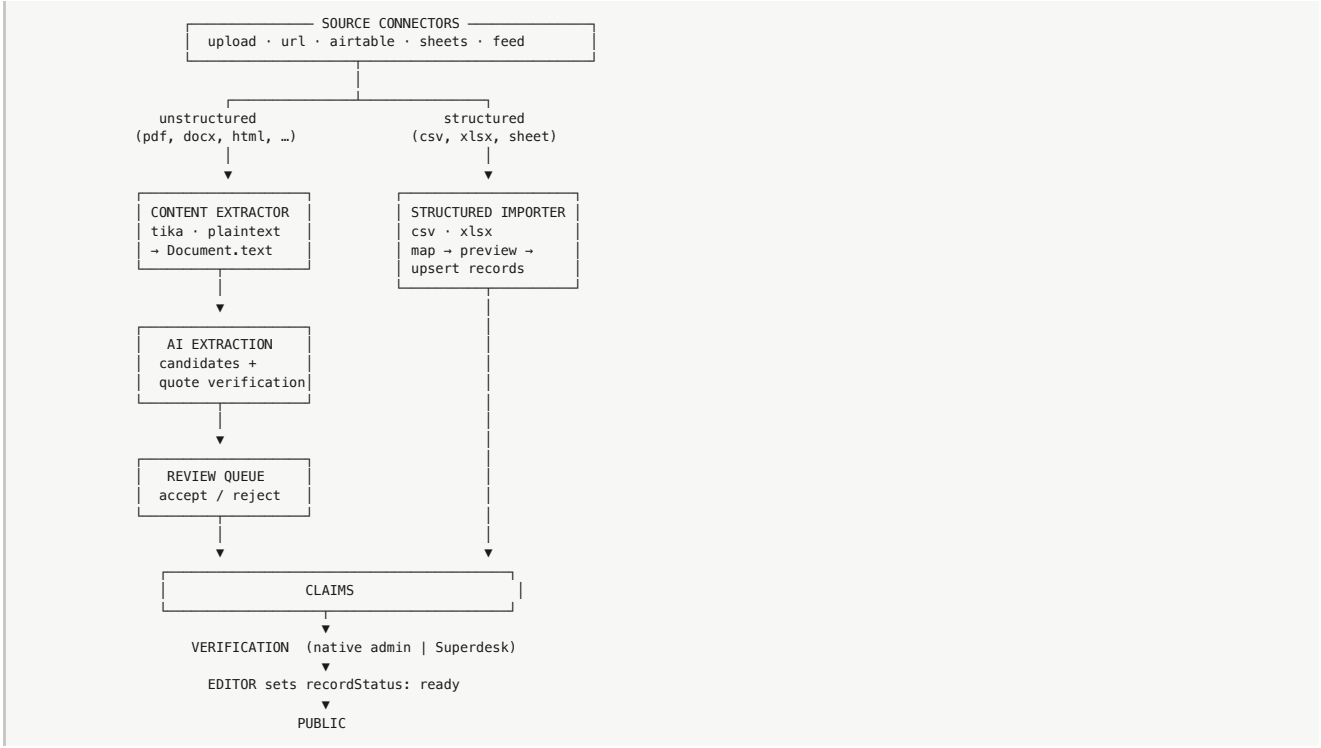
Status fields in the admin are the primary, mandatory feedback layer. Email is secondary, and only where a human can act.

Event	Notify?	Why
Any record moved to `archived`	Yes	May warrant a hard delete, and downstream work should be reviewed
Country moved to `hold`	Yes	Hides everything beneath it, and is easy to do by accident
Actor or document moved to `hold`	No	Routine, and visible in the admin's draft list
`archived` back to `ready`	Yes	Un-archiving is deliberate and should be acknowledged
Document processing failed after the retry budget	Yes, to the uploader	They can fix the file or the link
Extraction flagged `needs_review`	In-app only	Visible in the review queue
New submission	Digest, to moderators	Volume-dependent, so not per item
Infrastructure failure	No email	Goes to Sentry, and there is nothing an editor can do

5. Pipeline

From "a partner emailed us a manifesto" to "it's on the site", and every way in between.

5.1 Overview



The structured path skips text extraction, AI and the review queue. A CSV of promises a partner already compiled needs no language model to interpret it (D18).

5.2 Source connectors

Where content comes from. Each one is a `SourceConnector` adapter (§2.2).

Connector	Mode	Plugin	What it does
`upload`	manual	core	An editor uploads a file in the admin. Always available
`url`	manual	core	An editor pastes a URL. The backend fetches it through the SSRF guard (§8.4), detects the type and stores the artefact
`airtable`	pull	airtable	Polls a base on a schedule. Optional, and nothing in core depends on it
`sheets`	pull	sheets	Google Sheets as a structured source
`feed`	pull	feed	RSS and Atom monitoring of a publisher. Future

`Document.sourceUrl` stays attribution only and the backend never fetches it. The `url` connector is the one deliberate, audited exception. An editor explicitly asked for that URL, the request goes through the same hardening as every other outbound fetch, and the result is stored as a `Media` artefact so it is never fetched again.

Type detection

The connector reports a MIME type and the pipeline does not trust it. Detection runs in order.

1. Magic-byte signature, using `src/utils/fileSignature.ts` carried over from v2.
2. `Content-Type`, for the `url` connector.
3. File extension, last.

If the signature and the declared type disagree, the document is marked `failed` with a clear message. This is already how `/api/airtable-upload` validates uploads in v2, and the rule carries forward to every connector.

5.3 Content extraction

MIME type selects the `ContentExtractor` adapter, most specific first.

Extractor	Plugin	Accepts
`plaintext`	core	`text/plain`, `text/markdown`, `text/html` with readability extraction
`tika`	tika	`application/pdf`, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB, and the rest of Tika's roughly 1,000 formats

Extractor	Plugin	Accepts
`ocr`	ocr	Scanned PDFs and images, when the text layer is empty. Future

Core alone handles plain text and HTML, which is enough for a working system with no Tika container. Adding `plugin-tika` adds everything else, which is how ingest from docs, Excel, CSV and URLs is satisfied for the unstructured half.

An extractor returns Unicode-normalised (NFC) plain text, a page or section count where it knows one, and any document metadata it recovered, such as title, author and date, as suggestions an editor may accept.

Extractors normalise and collapse whitespace. They discard no characters, unlike v2's `cleanText` (§2.7).

Excel and CSV are ambiguous. A spreadsheet may be a document, such as a budget table to extract promises from, or a dataset whose rows are promises. The editor chooses at upload time. The default is the structured path for CSV and the document path for XLSX, and either can be switched before processing starts.

5.4 Processing state machine

`Document.processing.status` is the single source of truth and the work queue (§3.7).

```
pending → fetching → fetched → extracting → extracted → analysing → analysed → completed
   │                                                                              ▲
   └──────────────────── imported (structured path) ──────────────────────────────┘

any state → failed          any state → needs-review
```

Every task writes the status before and after its operation, so a document's position is always visible in the admin, including to the person who uploaded it. v2 could only deliver that feedback by writing strings into Airtable cells.

```
async function runStage<T>(doc: Document, stage: ProcessingStatus, fn: () => Promise<T>) {
  await setStatus(doc.id, stage, { startedAt: new Date() })
  try {
    const result = await fn()
    await setStatus(doc.id, doneStateFor(stage), { completedAt: new Date() })
    return result
  } catch (err) {
    await setStatus(doc.id, 'failed', { message: describe(err), lastAttemptAt: new Date() })
    throw err
  }
}
```

5.5 AI extraction

Owned by `plugin-ai`, and optional. With the plugin disabled, documents stop at `extracted` and claims are entered by hand.

The algorithm is carried over from v2 unchanged in substance. It is good, and its hallucination guard is the most valuable thing in the repository.

Pass 1, per-chunk extraction. The text is split into overlapping chunks on paragraph and sentence boundaries. Each chunk goes to the selected `LLMProvider` with a structured-output schema of `{ category, summary, sourceQuotes[] }`. The prompt insists on specific verifiable future commitments, forbids merging two commitments into one, and requires exact quotes in the document's own language.

The guard. Candidates are deduplicated, then every candidate whose quotes do not literally appear in the normalised document text is dropped. This is `hasVerifiableQuote` and `mergeByAtomicPromise` in `src/tasks/extractPromises.ts:189-213`. It is the difference between an assistant and a fabrication engine, and it must survive any refactor.

Pass 2, normalisation. A tool-forced pass splits compound commitments into atomic ones and titles the set, then runs through the same verification again.

Persistence. Results become one `Extraction` with its `candidates[]`, all `decision: 'pending'`. Nothing becomes a claim without a human.

What is dropped from v2

Dropped	Why
The 14-provider configuration matrix in the admin	Becomes `LLMProvider` adapters, and a deployment enables the one or two it uses
Per-document cost budgeting (`documentBudget.ts`, `maxCostUsdPerDocument`, per-million-token pricing in settings)	Nine tunables that encode pricing which goes stale. Replaced by one concurrency limit, one per-call timeout and one per-document token ceiling
`ai-extraction-export-rows`	A 28-column denormalised table maintained by hooks on five collections, existing only to produce a CSV. Replaced by an export that queries live (§7.6)

Review queue

The screen reviewers live in. For one document, each candidate appears with its proposed title, category and confidence, and with its verified quotes highlighted in the source text. The actions are Accept, Edit and accept, and Reject with a reason.

Accepting creates a `Claim` with `enteredBy: 'ai'`, `recordStatus: 'hold'` and `verification.verdict: 'unverified'`, linked back to the extraction and candidate. Rejecting records the reason, which is how extraction quality becomes measurable instead of anecdotal.

5.6 Structured import

For CSV, XLSX and sheets, rows become records directly (D18). The flow is map, preview, commit, never a blind upload.

1. **Inspect.** The importer reads headers and the first rows and returns a `TableShape`.
2. **Map.** The editor maps columns to fields. Mappings are saved as reusable profiles, so a partner who sends the same shape monthly maps once.
3. **Preview.** The import runs with no writes and reports rows to create, rows to update, rows to skip, and per-row validation errors with row numbers. An `ImportJob` record holds the result.
4. **Commit.** Only after the preview is accepted.

Target shapes are claims (with promisor and category resolved by slug or name), actors, claim updates for bulk-loading a history, and documents as metadata plus a source URL.

Each target declares a natural key: `(country, slug)` for claims and actors, `(claim, date)` for updates. Re-importing the same file updates rather than duplicates. An unresolvable reference, such as a promisor name that matches nothing, is reported as a row error instead of silently creating a stub record.

This is also how any curated list still sitting in Airtable comes in at launch, as ordinary day-one data entry rather than a migration (§9.3).

5.7 Verification

The claim goes to whichever `VerificationProvider` is active (§6). Either way the outcome is the same three fields on the claim, `verification.verdict`, `verifiedAt` and `rationale`, and the claim stays in PromiseTracker.

Verdicts of `false` and `misleading` are never published and never auto-deleted. They are the audit trail that makes extraction quality and systematic prompt failures visible. Archiving one is a deliberate admin action.

5.8 Publishing

Verification and publication are two separate steps, on purpose.

```
verdict = verified           →  the claim is true to the source
recordStatus = ready         →  the editorial content is fit to publish
both, plus ancestry          →  _status = published           (§4.2)
```

A verified promise with a half-written summary should not be live. Collapsing the two into one switch would make that impossible to express.

5.9 Resilience

Retries

Error	Retry	Fallback
Network timeout	3 times, exponential backoff	Next item, and alert above a 10% failure rate
Rate limit (429)	Yes, honouring `Retry-After`	Requeue, and alert if sustained over 5 minutes
Service unavailable (503)	3 times	Skip, and the scheduled sweep retries
Invalid data (400)	No	Mark `failed` with a specific message
Auth error (401 or 403)	No	Fail immediately and alert, because a credential is wrong
Server error (500)	2 times	Next item

Backoff starts at 1s, doubles, and caps at 30s.

Timeouts

Operation	Timeout
File fetch	5 min total, 30s per redirect hop
Text extraction	2 min
One AI call	2 min
One document's AI stage	20 min
Verification provider call	1 min
Health check	5s

Stuck jobs

A document stuck in a transient state has failed outside the application's control, through a restart, a network stall or a hung dependency, and gets its own recovery path.

Transient state	Considered stuck after
`fetching`	10 min
`extracting`	5 min
`analysing`	25 min
any other	2 min

A sweep runs every 5 minutes. `retryCount` is one budget shared by thrown errors and stuck sweeps, so a document cannot ping-pong between the two forever. Under 5, the sweep re-enqueues at the start of the current stage and increments. At 5, it marks the document `failed` with the last known cause and stops sweeping it.

Circuit breakers and rate limits

Every external client, including Tika, the selected LLM provider, Superdesk and any connector, is wrapped in a breaker: open after 5 consecutive failures, retry after 60s, 3 probe requests before closing. Per-provider concurrency and minimum-interval limits are configured per adapter rather than globally.

Idempotency

- **Inbound pushes** carry a delivery id, checked against the last-seen id on the target record before processing. v2's `/api/meedan-sync` already does this with a SHA-256 body fingerprint and an in-flight 409, and the mechanism carries over.
- **Record creation** relies on database unique indexes rather than application-level checks. On a duplicate-key error the handler falls back to update. No distributed locks and no `SELECT FOR UPDATE`, because the database serialises conflicting inserts atomically.
- **Files** deduplicate on SHA-256 checksum, and two documents may legitimately reference one `Media` item.
- **Scheduled workflows** take an atomic lock (`src/lib/workflowLocks.ts`), so an overlapping run skips rather than doubles up.

5.10 Tasks

Task	Plugin	Retries	Trigger
`pullSource`	per connector	3	Schedule, pull connectors only
`fetchDocument`	core	2	On document create with a pending fetch
`extractText`	core	2	On `fetched`
`extractCandidates`	ai	2	On `extracted`, when AI is enabled
`runImport`	core	0	On import commit
`submitForVerification`	verification adapter	3	On claim create, if the adapter pushes
`pullVerdicts`	verification adapter	3	Schedule, for reconciliation
`recomputeClaimStatus`	core	3	On claim-update change
`cascadeStatus`	core	3	On `recordStatus` change (§4.3)
`sweepStuck`	core	1	Every 5 min
`cleanupJobs`	core	1	Hourly

Every task is wrapped in the Sentry tracing helper carried over from `src/tasks/utils.ts`, so each one has a span, a duration, structured start, complete and error logs, and the ids needed to trace one document through the whole pipeline.

---

6. Plugins

One section per adapter. Each states which ports it implements, which fields it owns, what it injects, what it exposes, and how it fails.

6.1 plugin-core

Implements `VerificationProvider:native`, `SourceConnector:upload`, `SourceConnector:url`, `ContentExtractor:plaintext`, `StructuredImporter:csv` and `StructuredImporter:xlsx`.

Owns every collection and global in §3.

Core is the baseline, and the baseline is a complete product (D4). It is what CI runs against, what a new contributor gets from `pnpm dev` with no credentials, and the fallback when an integration is retired.

verification:native

Review inside the Payload admin. A reviewer opens a claim and sees the statement, the source document with the quote highlighted, the actor and the extraction provenance, then sets `verification.verdict` with a rationale. A saved view called "Awaiting verification" is the queue.

```
const nativeVerification: VerificationProvider = {
  id: 'native',
  async submit() { return null },        // nothing leaves the system
  async pull() { return [] },            // nothing comes back
  async health() { return { healthy: true, latencyMs: 0 } },
}
```

The null implementation is the point. With `native` selected, `verification.verdict` is an ordinary editable field and no synchronisation exists.

6.2 plugin-superdesk (D1)

Implements `VerificationProvider:superdesk`. Owns `verification.verdict`, `verification.verifiedAt` and `verification.rationale` on `claims`, and injects a read-only `superdesk` group there (§2.5).

Superdesk Core is Sourcefabric's newsroom production system. It has no native fact-check or claim model, so the adapter builds one out of a content profile and custom vocabularies. That is the idiomatic way to extend Superdesk, but the newsroom has to configure its instance before the integration can work, and several details below are instance-specific.

> Everything marked **CONFIRM** must be checked against the actual instance. Superdesk is mid-way through an async rewrite, and its published changelog stops at 2.4.5 in 2022. Pin the version in deployment configuration and re-verify.

Superdesk's three APIs

API	Prefix	Methods	Use
Internal	`/api` (`URL_PREFIX`)	full CRUD	The only write path. Creating claim items, patching, publishing
Production API	`/prodapi/v1` (`PRODAPI_URL_PREFIX`)	GET only	Reading items in any state, including unpublished
Content API	deployment-specific	GET only	Published items, NINJS-shaped

CONFIRM the Content API prefix. It derives from `CONTENTAPI_URL`, and Sourcefabric's own compose file yields `/capi/items` while other deployments yield `/v1/items`. Do not hardcode `/v1`. Confirm too that the Production API is deployed at all, since it is a separate process and many installations do not run it.

Authentication

API	Mechanism
Internal	`POST /api/auth_db` with service-account credentials returns a session token, sent as `Authorization: Basic base64("<token>:")`. Sessions expire after `SESSION_EXPIRY_MINUTES`, default 240, and the adapter re-authenticates on 401
Production API	OAuth2 client credentials for a JWT. `POST /api/auth_server/token` with `grant_type=client_credentials`, then `Authorization: Bearer <jwt>`, scope `ARCHIVE_READ`. Registered server-side with `manage.py auth_server:register_client`
Content API	A subscriber token, the `_id` of a `subscriber_token` document created via `POST /api/subscriber_token`

A service user is required. The auth server grants read-only scopes and has no write scope, so creating items needs a real Superdesk user account with the `archive` privilege. Treat its credentials as a privileged secret (§8), scope the account to the fact-check desk, and name it recognisably, such as `promisetracker-bot`, so its edits are attributable in Superdesk's own history.

Modelling a claim in Superdesk

A content item is a flat JSON document close to IPTC NINJS. The adapter uses:

PromiseTracker	Superdesk	How
Claim identity	`guid`	Superdesk-generated, stored back as `superdesk.guid`
`title`	`headline`	
`statement`	`body_html`	The verbatim quote in a blockquote, plus context

PromiseTracker	Superdesk	How
`summary`	`abstract`	
`type`	`subject[]` with `scheme: 'claim_type'`	Custom vocabulary
`category`	`subject[]` with `scheme: 'claim_category'`	Custom vocabulary
`verification.verdict`	`subject[]` with `scheme: 'verdict'`	Custom vocabulary. **CONFIRM** the scheme id
`verification.rationale`	`body_html` of the published fact-check	
Claim id, as a back-reference	`extra.promisetracker_id`	Extra field, `field_type: 'text'`
Claim URL	`extra.promisetracker_url`	Extra field
Language	`language`	
Item kind	`profile`	A content profile such as `fact_check`. **CONFIRM** the id

The verdict is a vocabulary rather than a text field because Superdesk's two extension mechanisms behave differently.

A custom vocabulary with `field_type: null` plus a `service` puts the selected value into the item's `subject` array as `{ qcode, name, scheme }`. It is a controlled list, it survives into NINJS, and it is queryable in Elasticsearch.

A vocabulary with `field_type` set becomes an extra field, stored at `extra.<cv_id>`. The `extra` dict is mapped `dynamic: false` in Elasticsearch, so filtering on `extra.*` may not work at all without a mapping change on the instance.

So anything the adapter needs to query goes in `subject` with a scheme, and anything it only needs to carry goes in `extra`. The PromiseTracker id is carried and never queried on the Superdesk side, because the adapter matches on `guid` locally.

CONFIRM the vocabulary ids, their `qcode` values, the content profile id, and the desk and stage ids. The adapter validates all of these at boot and refuses to start on a mismatch, rather than silently writing items nobody sees.

Outbound: creating a claim for review

```
POST /api/archive
Authorization: Basic base64("<session-token>:")
Content-Type: application/json

{
  "type": "text",
  "profile": "<fact_check profile id>",
  "headline": "Ruto: 'We will build 200 new health centres by 2027'",
  "body_html": "<blockquote>…verbatim statement…</blockquote><p>Source: 2022 manifesto, p.41</p>",
  "abstract": "…",
  "language": "en",
  "subject": [
    { "qcode": "unverified",       "name": "Unverified",       "scheme": "verdict" },
    { "qcode": "campaign-promise", "name": "Campaign promise", "scheme": "claim_type" },
    { "qcode": "health",           "name": "Health",           "scheme": "claim_category" }
  ],
  "extra": {
    "promisetracker_id":  "6745abc…",
    "promisetracker_url": "https://promisetracker.org/ke/promises/200-health-centres"
  },
  "task": { "desk": "<desk id>", "stage": "<stage id>" }
}
```

Superdesk fills in `guid`, `_id`, `family_id`, `unique_id`, `version` and `original_creator`, and sets `state: 'draft'`.

`task.desk` and `task.stage` are not optional in practice. Without them the item lands in the service account's personal workspace and no fact-checker will ever see it. Both ids are configuration, validated at boot.

Post to `/api/archive`, not `ingest`. Archive puts the item inside the newsroom at `state: 'draft'`, which is what an AI-extracted promise awaiting review means. The `ingest` route is the alternative when the newsroom wants its own routing rules, but a fetched ingest item gets a new guid, which breaks the stable identifier the adapter depends on.

Inbound: receiving verdicts

Two paths, and the adapter runs both.

The primary path is an HTTP push on publish. Configure a Superdesk subscriber with an `http_push` destination.

```
{
  "name": "PromiseTracker",
  "is_active": true,
  "subscriber_type": "all",
  "destinations": [{
    "name": "promisetracker",
    "format": "ninjs",
    "delivery_type": "http_push",
    "config": {
      "resource_url": "https://promisetracker.org/api/plugins/superdesk/items",
      "assets_url":   "https://promisetracker.org/api/plugins/superdesk/assets",
      "secret_token": "<shared secret>"
    }
  }],
  "products": ["<product id>"]
}
```

Superdesk `POST`s the NINJS item and signs it.

```
x-superdesk-signature: sha1=<hex HMAC-SHA1(secret_token, raw_body)>
```

The handler must verify this signature in constant time before parsing. HMAC-SHA1 is weak by modern standards but it is what Superdesk implements, so compensate with the secret's entropy and treat the endpoint as untrusted input regardless.

The secondary path is a reconciliation poll. Every 15 minutes a job pulls items changed since the last cursor and reconciles. Pushes are not guaranteed, since a deploy, a network blip or a disabled subscriber loses them silently. This is the same webhook-plus-safety-poll argument the v2 reference architecture made for Airtable.

```
GET {contentApiPrefix}/items
      ?filter={"range":{"versioncreated":{"gte":"2026-11-06T10:00:00+0000"}}}
      &sort=[("versioncreated",1)]
      &max_results=200
Authorization: Basic base64("<subscriber-token>:")
```

Polling, precisely

The Content API is unusually constrained, and getting this wrong produces a sync that looks like it works and quietly misses items.

- `start_date` and `end_date` are date-only (`YYYY-MM-DD`), inclusive, and cannot be in the future, which makes them useless for a 15-minute window. Use the `filter` parameter with a raw Elasticsearch range on `versioncreated` instead.
- With no dates given, the default window is the last 7 days. Anything older is invisible, so a backfill job must walk date ranges explicitly.
- There are no ETags. `_etag`, `_created` and `_updated` are stripped from Content API responses, so conditional requests are unavailable and the sync cursor must be `versioncreated`.
- Query parameters are a strict whitelist. Anything outside it returns an error rather than an ignored parameter.
- Pagination defaults to 500 and caps at 1,000.
- Overlap the window by 10 minutes. `versioncreated` is server-side UTC and Elasticsearch has a refresh interval, so an item can become visible after a poll whose window should have included it.
- Store the cursor as the maximum `versioncreated` actually observed, never wall-clock time.
- Date formats differ between APIs. The internal and Content APIs use `+0000`, and the Production API defaults to RFC 3339 `+00:00`. Parse both.
- Visibility is subscriber-scoped. The Content API filters by a `subscribers` array written at publish time, so if PromiseTracker's subscriber matches no product the API returns `200` with an empty list. That is a silent no-data failure, not an error. Test it explicitly, and treat a sustained empty result as a health-check failure rather than as nothing new.

Item state mapping

Superdesk reuses one `guid` across corrections and kills, with a new `version` and a new `versioncreated`. There is no delete.

Superdesk `state`	**PromiseTracker effect**
`draft`, `in_progress`, `submitted`	In review. No change to the claim
`published`, `corrected`	Apply the verdict and rationale, and set `verifiedAt`
`killed`, `recalled`, `takedown`	Depublish: `verification.verdict` to `unverified`, `recordStatus` to `hold`, and notify an editor
`unpublished`	Same as above
`spiked`	`recordStatus` to `archived`
`scheduled`, `being_corrected`, `correction`	No change. Wait for the terminal state

Note the exact strings. It is `in_progress`, not `progress`, and `pubstatus withheld` is represented by a constant named `HOLD`. These are easy mismatches to ship.

Dedupe on `(guid, version)`. A correction republishes the same `guid` with a higher version and that is new information, while the same `(guid, version)` twice is not. The publish queue retries and does not guarantee ordering, so the handler must also tolerate out-of-order arrival and apply only when the incoming version is greater than the stored one.

The adapter both creates items and receives them, so it guards against loops. Every inbound item carrying an `extra.promisetracker_id` the system already owns is matched to that claim and never re-created. An inbound item with no `promisetracker_id` is ignored by default, and importing newsroom-originated claims is a separate behaviour that has to be enabled.

What is not used

- Websocket notifications on port 5100 carry only resource ids and changed field names, no content, and have no authentication. They are an internal component, normally not exposed, and not a dependency.
- `POST /api/webhook` only re-triggers an ingest provider poll. It does not receive content.
- Content-list webhooks exist on `develop` but are not in the published documentation. **CONFIRM** before designing around them.

Failure behaviour

Failure	**Behaviour**
Boot with an invalid profile, vocabulary, desk or stage id	Refuse to start, naming the bad id

Failure	Behaviour
Session expired (401) on write	Re-authenticate once, retry once, then fail the task
Push signature invalid	Return `401`, log at `warn` with the source IP, do not parse the body
Push for an unknown `guid`	Return `200` so Superdesk stops retrying, and record for reconciliation
Superdesk unreachable	Circuit breaker opens, claims queue locally, health check reports `degraded`
Content API empty for 3 consecutive polls	Health check `degraded`, since a subscriber or product misconfiguration is likely

Nothing about PromiseTracker stops working when Superdesk is down. Claims are still created, extracted, edited and published. Only new verdicts stop arriving. That is the whole point of the port.

6.3 plugin-check, specified but not built (D6)

Meedan Check is documented here as a second implementation of `VerificationProvider`, to show the interface is not Superdesk-shaped. It is not built in v3.

Port method	Check equivalent
`submit`	`createProjectMedia` GraphQL mutation, `media_type: "Claim"`
`pull`	`search(query: { report_status: ["published"] })`
`receive`	`POST /api/plugins/check/webhook`, `annotation_type: "report_design"`

Mapping Check onto the same three verdict fields needs no change to the interface, which is the result worth knowing. The v2 implementation, roughly 2,000 lines across `src/lib/meedan.ts`, `src/lib/syncMeedanReports.ts`, three tasks, a workflow and a webhook route, is not carried forward, and with no Check data to migrate the workspace is simply retired (§9.4).

One v2 lesson: `syncMeedanReports` could not recover a promise's political entity or category from a Check report, so it rebuilt an in-memory index of every `ai-extractions` document on every run. The port avoids that by keeping the claim in PromiseTracker and letting only the verdict cross the boundary.

6.4 plugin-ai

Implements `LLMProvider` for each configured vendor. Owns the `extractions` collection, the `extractCandidates` task and the review queue UI. It injects nothing on `documents`, where it reads `text` and writes `processing.status`.

Adapters wrap the Vercel AI SDK provider factories. The catalogue, model-id resolution and credential validation come straight from v2's `src/lib/ai/providerCatalog.ts` and `providerRegistry.ts`, which already do this job behind a hardcoded switch.

```
PT_LLM_MODEL=google:gemini-2.5-pro
```

Credentials live in Settings under AI, encrypted. One provider is selected for extraction, and others may be registered and are available for a future second opinion.

Health is a cheap completion against the selected model at startup and on `/api/health`. Exhausted credits surface as `degraded` rather than as a flood of failed documents, a failure mode v2 only discovered through Sentry.

6.5 plugin-tika

Implements `ContentExtractor:tika`.

A thin client over a self-hosted Apache Tika server. `docker compose up -d` already provides one in v2, and the production image bundles Tika 3.2.3.

- Accepts PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and the rest of Tika's format list, which is most of what ingest from docs, Excel, CSV and URLs asks for.
- Returns NFC-normalised text, a page count, and document metadata as editor suggestions.
- Two-minute timeout, circuit breaker, and a participant in `/api/health`.
- Required for the health check only when enabled. In v2, Tika is an unconditional readiness dependency, so a Tika outage marks the whole app unready even though the public site never touches it. In v3 the public read path never depends on an extractor.

6.6 plugin-airtable, optional and not required

Implements `SourceConnector:airtable` and `StructuredImporter:sheets`. Owns nothing in core unless explicitly enabled to.

Retained as an option for teams still working in Airtable, not as a dependency. The one real difference from v2 is the direction of authority. With the plugin enabled, Airtable is a source of documents and actors and may take ownership of those fields, but it is never the work queue and never the operator console. Processing state lives in `Document.processing.status` and is visible in the admin whatever else is running.

For most teams the better exit is a one-off structured import (§5.6) rather than a standing sync.

6.7 Plugin routes

App Router routes are filesystem entries and cannot be registered at runtime, so plugin routes follow a convention and are declared in the manifest.

```
src/app/api/plugins/
├── superdesk/
│   ├── items/route.ts      POST, receives pushed NINJS items
│   └── assets/route.ts     GET and POST, media exchange
└── airtable/
    └── webhook/route.ts    POST
```

Every handler's first action is a plugin check. If the plugin is not enabled it returns `503` immediately, without executing any application logic. The manifest's `routes[]` declares the path, methods, purpose and the secret each one needs, which is how the integration tests discover what to exercise instead of hardcoding paths.

---

7. Public site

7.1 Routing

One site, one deployment, country as a path segment (D10). No subdomains, no tenant resolution, no `getDomain()`, no multi-tenant plugin.

```
/                                 Global home, country selector
/about, /methodology, /{pageSlug}   Global pages

/{cc}                             Country landing
/{cc}/politicians                 Listing, filterable
/{cc}/politicians/{slug}          Profile: scorecard, promises, map
/{cc}/parties                     Listing
/{cc}/parties/{slug}              Profile
/{cc}/institutions                Listing
/{cc}/institutions/{slug}         Profile
/{cc}/places                      Listing
/{cc}/places/{slug}               Profile: map, promises in and to this place
/{cc}/promises                    All claims, filterable and paginated
/{cc}/promises/{slug}             Claim page: statement, evidence, timeline, map
/{cc}/search                      Search results
/{cc}/{pageSlug}                  Country page (about, methodology, partners)
```

`cc` is `Country.code`, the lowercase ISO 3166-1 alpha-2 code: `/ke`, `/za`, `/ng`.

Typed segments (D15)

Each actor kind has its own segment, so a slug need only be unique within `(country, kind)`.

Kind	Segment
`person`	`politicians`
`party`	`parties`
`institution`	`institutions`
`place`	`places`

`/ke/politicians/nairobi` and `/ke/places/nairobi` can coexist. A flat `/ke/{slug}` would force global slug uniqueness across every kind and require a lookup across all of them before the server knows what it is rendering. That is what v2 does, and why its catch-all route runs to 443 lines with four near-identical branches.

Slug validation on `Pages` reserves the first segments under a country: `politicians`, `parties`, `institutions`, `places`, `promises`, `search`, `api`, `embed`, `sitemap.xml` and `feed`.

Resolution

The country is the first segment. Middleware resolves it once per request from a cached map of code to id, a handful of records cached indefinitely and invalidated on country change, then passes it down through the request context. Blocks and components receive the resolved country and actor, and never re-query by slug.

That rule comes from v2's `src/lib/routes/publicRoutes.ts`, where it prevented cross-tenant slug collisions. Here it is also a performance rule: v2 re-ran `getTenantBySubDomain()`, which itself calls `getAllTenants()`, from the layout, `generateMetadata`, the page body and five block components, so one render re-queried the tenant list half a dozen times.

An unknown country code returns 404. No fallback, no redirect to a default.

7.2 Caching

The single biggest performance lever. v2 sets `export const dynamic = "force-dynamic"` on every public route and caches nothing at all: no `unstable_cache`, no `revalidate`, no `generateStaticParams`, no ISR.

v3 caches by tag and invalidates precisely.

Tag	Invalidated when
`country:{code}`	The country record changes
`actor:{id}`	That actor, or a claim naming it, changes
`claim:{id}`	The claim or any of its updates changes
`claims:{country}`	Any claim in that country is published or unpublished
`statuses`, `categories`	The taxonomy changes
`settings`	Site settings change

- Page data is wrapped in `unstable_cache` with the relevant tags and a long `revalidate` ceiling as a backstop.
- `generateStaticParams` prerenders country landings, actor profiles and claim pages for published records. The long tail renders on demand and is then cached.
- The cascade job calls `revalidateTag` on completion (§4.3), so publishing propagates in seconds rather than waiting for a timer.

- Query `depth` is specified per query. v2 uses `depth: 6` on page loads, which walks relationships six levels deep on every render.
- Media is served with `Cache-Control: public, max-age=31536000, immutable`.

The election-day target is a country landing and a claim page served from cache in under 100ms at the origin, with the CDN absorbing the rest.

7.3 Search

v2 renders a search box in `src/components/Search.tsx` with no handler, no route and no index. v3 ships real search.

- MongoDB text indexes over `claims` (`title`, `statement`, `summary`) and `actors` (`name`, `shortName`, `description`), weighted toward titles and names.
- Scoped by country by default, with an all-countries option.
- Results grouped by type: claims, politicians, parties, institutions, places.
- Server-rendered at `/{cc}/search?q=`, so results are linkable and crawlable.
- Isolated behind one module, so moving to Atlas Search or Elasticsearch later touches a single file (§1.6).

7.4 Listings and filtering

v2 loads every published promise for an entity with `pagination: false`, derives facets in the browser, and filters client-side. At 50 promises that is fine. At 5,000 it is a multi-megabyte payload. v3 instead uses:

- Server-side pagination, 24 per page, cursor or offset.
- Server-side filtering on status group, category, actor, claim type and date range.
- Facet counts from aggregation rather than from the loaded page.
- Filter state in the URL, as in `?status=kept&category=health&page=2`, so a filtered view is shareable, bookmarkable and indexable, and the back button works.
- Sorting by most recently updated, deadline or status.

7.5 Pages

Claim page, `/{cc}/promises/{slug}`

The product's most important page.

1. **Header.** Status chip in the colour from the status record, claim type, category, date made, and target date if any.
2. **The statement.** The verbatim quote, visually distinct, attributed to the promisor, with a link to the source document and page.
3. **Context.** The editorial summary.
4. **Timeline.** Every published `ClaimUpdate` in reverse-chronological order: date, status at that date, narrative, evidence links. This is the feature v2 promises and does not have.
5. **Verification.** The verdict, who verified it and when, the rationale, and any published fact-check report.
6. **Map.** Where the claim applies, when `jurisdiction` or the promisor's office carries geodata (§7.8).
7. **Actions.** Share, and submit an update (§5).
8. **Related.** Other promises by the same actor in the same category.

Actor profile, `/{cc}/politicians/{slug}` *and siblings*

Scorecard first: total promises, and the count and share in each status group, rendered as a chart. Then office history for a person, current office, party, and a filterable promise list. A map shows where they are from (`person.originPlace`) and where they govern (`offices[].jurisdiction`), neither of which v2 could express, having only a free-text `region`.

Status grouping comes from `ClaimStatus.group` (§3.4), not from per-page block configuration.

Country landing, `/{cc}`

A country map with regions, headline numbers, featured politicians, recently updated promises, and entries into each actor listing.

7.6 Distribution

Embeds

v2 sets `frame-ancestors 'self'` in its CSP (`src/lib/security/headers.mjs:53`), so PromiseTracker pages cannot be embedded anywhere. v3 adds dedicated embed routes with their own policy, and the main site keeps `'self'`.

```
/embed/claim/{id}           A single promise card with status and last update
/embed/scorecard/{actorId}  An actor's status breakdown
/embed/list?...             A filtered promise list
```

- Served from `/embed/*` only, with `frame-ancestors *` on those routes alone.
- No cookies, no tracking, and no JavaScript beyond an auto-height postMessage.
- A `<script>` snippet handles sizing, and a plain `<iframe>` works without it.
- Each one carries a visible link back to the canonical page.

- An oEmbed endpoint at `/api/oembed`, so pasting a promise URL into a CMS produces the card.

Public read API

```
GET /api/public/v1/countries
GET /api/public/v1/{cc}/actors?kind=person&page=1
GET /api/public/v1/{cc}/claims?status=kept&category=health&updatedSince=2026-01-01
GET /api/public/v1/{cc}/claims/{slug}
GET /api/public/v1/{cc}/claims/{slug}/updates
```

- Published records only, under the same `_status` filter as the site, so public has one definition.
- Stable, documented JSON, not the raw Payload REST surface. v2 exposes `/api/[...slug]` with `read: publishedOrAuthenticated` on promises, which makes the internal shape the public contract and impossible to change without breaking consumers.
- Unauthenticated, rate-limited per IP, cached at the CDN.
- `Cache-Control` and `ETag` on every response.
- An OpenAPI description at `/api/public/v1/openapi.json`.
- Toggleable through `Settings.features.publicApiEnabled`.

Export

CSV export of claims and updates, filtered, generated on demand from a live query.

v2 maintains `ai-extraction-export-rows` for this: a 28-column denormalised table kept in sync by hooks on five collections plus a reconciliation job. v3 queries and streams.

7.7 SEO and discoverability

v2 has none of this: no sitemap, no `robots.txt`, no canonicals, no JSON-LD, no `metadataBase`, no OG images.

Feature	Implementation
Sitemaps	A sitemap index at `/sitemap.xml`, one child per country, split at 50,000 URLs
`robots.txt`	Generated, pointing at the sitemap index
Canonical URLs	On every page, absolute, from a configured `metadataBase`
OG images	Generated per claim and per actor through `next/og`, cached
ClaimReview JSON-LD	On every verified claim page
Organization, Person and Place JSON-LD	On actor profiles
Breadcrumbs	`BreadcrumbList` JSON-LD matching the typed URL structure
RSS and Atom	Per country, for recently published and recently updated claims

ClaimReview is schema.org's standard for fact-checks and what Google's fact-check surfaces consume. Emitting it makes every verified promise eligible for those surfaces and makes the data legible to AI assistants that read structured data. It is also the interoperability format the wider fact-checking community already uses, which matters more now that Superdesk is the review backend and Superdesk emits NINJS rather than ClaimReview.

```
{
  "@context": "https://schema.org",
  "@type": "ClaimReview",
  "url": "https://promisetracker.org/ke/promises/200-health-centres",
  "claimReviewed": "We will build 200 new health centres by 2027",
  "itemReviewed": {
    "@type": "Claim",
    "author": { "@type": "Person", "name": "William Ruto" },
    "datePublished": "2022-06-30"
  },
  "author": { "@type": "Organization", "name": "PromiseTracker" },
  "reviewRating": {
    "@type": "Rating",
    "ratingValue": 2, "bestRating": 5, "worstRating": 1,
    "alternateName": "In progress"
  }
}
```

The numeric rating derives from `ClaimStatus.group`, and `alternateName` is the localized label.

7.8 Maps (D17)

Where the promise applies, and where the politician is from.

- Rendered from `GeoRef` on the country, the place, or the promisor's office jurisdiction (§3.9).
- Boundary polygons are stored GeoJSON uploads, not live third-party geometry. A map that stops working because an external tile or boundary service changed its terms is exactly the coupling v3 exists to remove. Attribution and licence are required fields.
- Degradation is clean: boundary, then a marker at `centroid`, then no map. A country with no geodata shows no map and nothing else breaks.
- Client-side, lazily loaded, below the fold, excluded from LCP. The page is fully readable without it.

- Base tiles come from a configured provider with attribution. **CONFIRM** the choice and its licence for commercial and embedded use before implementation.

Maps appear on the country landing, optionally shading regions by promise performance, on a place profile with its own boundary and children, on a politician profile for origin and jurisdiction, and on a claim page for the jurisdiction it applies to.

7.9 Internationalisation

- Content locales `en` and `fr`, per country (§3.1).
- A visible language switcher, which v2 has nowhere despite shipping a bilingual admin and localized content.
- Locale as a query parameter or a cookie-backed preference, with `hreflang` alternates on every page.
- All UI strings localized. v2 hardcodes English filter and sort labels in `src/components/Promises/index.tsx` and leaves `promise-list.title` unlocalized.

7.10 Accessibility and performance budgets

Target	Budget
LCP	< 2.5s on 4G
CLS	< 0.1
TTI	< 3s
JS on a claim page	< 150 KB gzipped, excluding the lazily-loaded map
Contrast	WCAG AA, including every status chip colour
Keyboard	Every filter, timeline control and map control reachable
Screen readers	The timeline is a semantic ordered list, not a decorative SVG

Status colours are editor-configurable, so the admin shows a live contrast check against the chosen text colour and warns below AA. v2 lets an editor pick any pair of colours with no check at all.

---

8. Security and operations

Most of this carries over from v2, which did the work well and documented it in `docs/remote-file-ingestion.md` and `docs/security/embeds-and-csp.md`. Where v3 changes something, it says so.

8.1 Authentication and authorization

Surface	Auth
Public site and public API	None
Embed routes	None
Admin	Payload session
Plugin inbound routes	A per-route shared secret, compared in constant time
Internal job trigger	Payload session

§4.5 holds the roles and the permission matrix. Collection access functions enforce them, and field-level access guards `recordStatus`, `verification.*` and every plugin-owned field.

Superdesk needs a real user account rather than an API client (§6.2). Treat it as a privileged credential and rotate it on the same schedule as the rest.

8.2 Input validation

A schema validates every piece of external data before use: webhook payloads, API responses, import rows, form submissions. Nothing reaches a Payload write unvalidated.

Inbound push handlers follow a fixed order.

1. Verify the signature before parsing the body.
2. Check the idempotency key against what has already been processed.
3. Validate the payload shape.
4. Only then touch the database.

8.3 Secrets

Secret	Where	Rotation
`PAYLOAD_SECRET`, `DATABASE_URI`	Environment	Annually
S3 credentials	Environment	Annually
Plugin API keys and tokens	Settings global, encrypted at rest	On offboarding
Inbound webhook secrets	Environment	Every 6 months

Plugin credentials live in the database because they change per deployment without a redeploy. `src/fields/encryptedSecret.ts` carries over unchanged. It encrypts on write with a version prefix, returns a mask to non-local callers, decrypts only for the Local API, and preserves the stored value when the mask is resubmitted, which is what stops a careless admin save from wiping a key.

Never logged: keys, tokens, passwords, submitter email addresses, raw IP addresses.

8.4 Outbound requests (SSRF)

The policy is unchanged from v2 and is the strongest part of its security posture.

By default the backend makes no outbound request to a user-supplied URL. `Document.sourceUrl`, `Evidence.url` and `Submission.evidenceUrl` are attribution only. They are stored, displayed and linked, never fetched.

The one exception is the `url` source connector, where an editor has explicitly asked for a URL to be retrieved. It runs through the existing guard in `src/utils/ssrf.ts` and `src/utils/files.ts`.

- HTTPS only. Every other scheme is rejected.
- The hostname resolves before the request, and the request is rejected if any resolved address is private (`10/8`, `172.16/12`, `192.168/16`), loopback, link-local including the `169.254.169.254` cloud-metadata endpoint, CGNAT, multicast or reserved, along with the IPv6 equivalents `::1`, `fe80::/10`, `fc00::/7`, IPv4-mapped, NAT64 and 6to4. IP-literal URLs are checked directly.
- Redirects are followed manually, every hop is re-validated before it is dialled, and there is a hop limit.
- Timeouts apply per hop and in total. A byte cap is enforced from `Content-Length` and again while streaming, under a global concurrency limit.
- MIME validation plus magic-byte signature verification. SVG uploads are rejected outright.

One residual risk, stated plainly. DNS is validated immediately before each request, so a malicious authoritative server with a 0-TTL record could attempt a rebinding race in the window between validation and connection. The hosting environment's egress policy is the mitigation.

8.5 Uploads

- The extension and the declared MIME type must both be allowed and must agree with each other, and magic bytes must then match the claimed type.
- Size caps per kind.
- A rate limit and concurrency gate per client.
- A rolling daily byte budget, which bounds storage and bandwidth cost.
- Files are stored by content hash, so identical uploads deduplicate.

8.6 Public endpoint protection

Endpoint	Protection
Submission form	Captcha, per-IP rate limit, honeypot field, and a salted IP hash for abuse handling
Public API	Per-IP rate limit, CDN cache, pagination caps
Embeds	No cookies, no state, CDN-cached
Inbound plugin routes	Shared secret, per-IP rate limit, idempotency cache
Search	Query length cap, rate limit

Raw IP addresses are never stored. Submissions keep a salted hash, which is enough to detect abuse patterns and not enough to identify a contributor.

8.7 Content security policy

The main site keeps a strict policy, including `frame-ancestors 'self'`.

`/embed/*` is the deliberate exception, with `frame-ancestors *` on those routes alone, serving pages that carry no cookies, no authentication and no state. The split keeps the admin and the main site un-embeddable while letting partners syndicate (§7.6).

Extracted text and any HTML from an external source is sanitised before storage, with an allowlist of tags and no attributes.

8.8 Observability

Logs are structured JSON. Every entry carries level, message, timestamp, source such as `task.extractText` or `plugin.superdesk.push`, and the ids needed to trace one document end to end: `documentId`, `claimId`, `countryId`, `taskId`, `workflowRunId`, `plugin`. Errors carry code, stack and metadata, and any operation over 100ms carries a duration.

Three rules. Log to console and to Sentry. Log success as well as failure, because success logs are what make anomaly detection possible. Never log a secret.

Metrics in Sentry: documents processed by stage, extraction candidates proposed and accepted, verification round-trip time, cascade job duration, push deliveries received and rejected, public API request rate, cache hit ratio.

Health lives at `/api/health`.

```
{
  "status": "healthy | degraded | unhealthy",
  "timestamp": "…",
  "plugins": { "enabled": ["core","tika","ai","superdesk"],
               "ports": { "verification": "superdesk", "extractors": ["plaintext","tika"] } },
  "services": {
    "database":  { "healthy": true, "latencyMs": 4 },
    "storage":   { "healthy": true, "latencyMs": 22 },
    "tika":      { "healthy": true, "latencyMs": 31 },
    "llm":       { "healthy": true, "latencyMs": 180 },
    "superdesk": { "healthy": true, "latencyMs": 95 }
  }
}
```

Only enabled plugins appear. Two changes from v2 are worth noting.

The response reports the resolved plugin and port configuration, which is the first thing anyone wants when a deployment misbehaves.

The public read path has no external dependency. v2 makes Tika an unconditional readiness dependency, so a Tika outage marks the whole application unready even though no public page touches it. In v3, `/api/health/ready` covers only the database and storage, while extractors, AI and the verification provider report into `/api/health` and degrade the pipeline rather than the site.

8.9 Alerting

Condition	Route
Pipeline success rate below 90% for 15 min	Slack
A dependency unhealthy for over 5 min	PagerDuty
Error rate above 5% for 10 min	Slack
Push signature failures spike	Slack, since someone is probing or a secret rotated badly
Content API empty for 3 consecutive polls	Slack, since a subscriber or product misconfiguration is likely (§6.2)
AI credits exhausted	Slack, plus health `degraded`

Condition	Route
Database p95 over 500ms	Slack warning

Editors are never paged for something only an SRE can fix. A failure an editor can fix, such as a corrupt upload or a dead link, surfaces in the admin on the record itself.

8.10 Configuration validation

At boot, before serving:

1. Core environment variables present.
2. `PT_PLUGINS` resolves, dependencies satisfied, no cycles.
3. Every exclusive port has exactly one implementation.
4. `requiredEnv` present for each enabled plugin.
5. Database and storage reachable.
6. Each enabled plugin's `validateSettings()` runs. Superdesk checks its profile, vocabulary, desk and stage ids against the live instance.

Any failure prints every problem at once and exits non-zero. A half-configured deployment that starts and then fails per request is worse than one that does not start.

8.11 Deployment

- Node 24, pnpm, a single container, with Tika in its own container when enabled.
- MongoDB and S3 external.
- Zero-downtime rolling deploys, with the health check gating readiness.
- Schema migrations are versioned, reversible where possible, and run before the new version serves.
- Rollback triggers: error rate over 10% for 5 minutes, broken critical functionality, data corruption.

Backups. The database gets a daily full backup with 30-day retention on versioned object storage. Media gets versioning plus cross-region replication. RTO is 4 hours and RPO is 24 hours. Restores are tested quarterly, because an untested backup is a hypothesis.

8.12 Data retention

Data	Retention
Claims, updates, actors, documents	Indefinite, archived, never auto-deleted
Extraction `rawResponse`	90 days, then pruned. Candidates and decisions are kept
Rejected and spam submissions	180 days
Submitter contact details	Deleted on request, and never published without `consentToAttribute`
Completed jobs	Deleted on completion
Failed jobs	7 days
Logs	Per the Sentry plan

8.13 Testing

Layer	Share	Covers
Unit	around 80%	Pure functions: status computation, slug rules, quote verification, import mapping, retry and backoff
Integration	around 15%	Tasks with mocked externals, route handlers, access control, the cascade, plugin resolution
End-to-end	around 5%	Upload, extract, review, verify, publish, public page; and submission, moderation, timeline

Coverage targets are 70% overall, 85% on the pipeline, governance and plugin resolution, and 90% on utilities.

Every plugin is tested both enabled and disabled, and a disabled plugin must return the config byte-identical to what it received. Core is tested entirely on its own, with no credentials for anything. If that suite needs an API key, D4 has been violated and the build should fail.

The v2 security suite carries over as the regression floor:
`tests/int/{ssrf,fileSignature,requestGuards,securityHeaders,privilegedAccess,publicApiAccess,publicRoutes}.int.spec.ts`.

---

9. Launch

9.1 There is no data migration

PromiseTracker has never gone live. v2 holds no production content, and every record in v3 will be entered fresh. That one fact deletes most of what a section like this normally contains. There is no field mapping, no reconciliation, no dual-running, no freeze window, no redirect map, and no one-off migration scripts to write, test and then throw away.

Three much smaller things remain.

1. **Seeding** the handful of configuration records everything else hangs off, in the order they have to exist (§9.2).
2. **Retiring Airtable and Meedan Check as tools,** which means revoking credentials and deleting code, not moving data (§9.4).
3. **Deciding, table by table, whether anything curated in those tools is worth re-entering** as ordinary day-one content (§9.3).

Not in scope (D12): whether v3 is built by refactoring this repository in place or in a fresh one. That remains a separate technical decision. Worth noting, though, that with no data to carry across, the usual argument for refactoring in place, that the migration is cheaper if the schema evolves, no longer applies. The cost of either option is now almost entirely about code reuse (§2.7), not data.

9.2 What has to exist before anything else

An empty v3 is not immediately useful. A few collections are referenced by everything else and have to be authored first. This is the dependency order, and also the order a new country is onboarded in (§1.5).

Order	Collection	Who authors it	Why it must come first	Minimum to launch
1	`claim_statuses`	Editorial lead, once	Every claim update points at one, and `group` drives every scorecard	One full set, every record with a `group`
2	`categories`	Editorial lead, once	Claims filter and chart by category	8 to 15 categories, localized
3	`countries`	Super-admin	The first path segment. Nothing public resolves without one	One per country at launch
4	`users`	Super-admin	Roles gate every other step	One super-admin, one editor per country
5	`site_settings`	Super-admin	Navigation, footer, branding	Enough for a page to render
6	`actors`	Editor, per country	Claims need a promisor	The politicians being tracked at launch

Only then do documents, extractions and claims make sense, and those follow the ordinary pipeline (§5) rather than any special import path.

`ClaimStatus.group` is the one that bites. It has no obvious default and no upstream source, and scorecards come out wrong rather than empty if it is missing or inconsistent. Assign it deliberately, once, before the first claim is published, not retrospectively (§10.3, question 12).

9.3 Content the old tools hold

Not live is not the same as empty. The Airtable base and the Check workspace hold work people actually did, and each table deserves an explicit keep-or-drop decision rather than quiet abandonment.

Where	What it is	Recommendation
Airtable `CSO Promise Updates`	Crowd-sourced promise updates, prefilled by the embedded public form. No v2 code has ever read this table	Triage by hand into `submissions` or `claim_updates`, or drop it as a decision. This is contributed content, and losing it silently is the bad outcome
Airtable entity and country tables	Curated lists of politicians, offices and regions	Re-enter, or import as CSV. Note that v2's sync skipped any entity without an image or term dates, so Airtable is the fuller list, not v2
Airtable document rows	Source manifestos and their URLs	Re-upload the files and re-enter the metadata
Check workspace	Claims and reports created during pilots	Review for anything worth keeping. Verdicts do not transfer, because a v3 claim is verified in v3

Where a list is long enough to be worth not retyping, the route in is the structured importer (§5.6), using the same mapping, preview and commit flow a partner newsroom's spreadsheet uses. That is deliberate. It means this work needs no bespoke migration code, and it exercises the importer before an outside party does.

Nothing arriving this way carries an `airtableID`, a `meedanId` or a `checkMediaId`. Those fields do not exist in v3 (§3).

9.4 Retiring the integrations

With no data to reconcile, both integrations are removed rather than wound down.

Airtable

1. Decide the fate of each table (§9.3) and import anything being kept.
2. Revoke the API key.
3. Delete `/api/airtable_upload` and its bearer token, the Airtable tasks and workflow, the generated typings, `pnpm generate:airtable`, and the `settings.airtable` group.
4. Archive the base read-only rather than deleting it, for a few months.

Teams that genuinely want to keep working in Airtable can run `plugin-airtable` instead (§6.6). It is an option, not the default, and either way the work queue no longer lives there.

Meedan Check

1. Revoke the API key.
2. Delete `/api/meedan-sync`, `WEBHOOK_SECRET_KEY`, `src/lib/meedan.ts`, `src/lib/syncMeedanReports.ts`, the three sync tasks and their workflow, and the `settings.meedan` group.
3. Delete `scripts/dedupe-meedan-promises.ts` and the `checkSyncMetadata` global.
4. Drop `promises.meedanId` and `ai-extractions.checkMediaId` with the collections themselves.

The `meedanId` hazard that script exists for, where `meedan-sync` only ever backfills a missing `meedanId` and never overwrites one because overwriting orphaned records in the past, is a property of v2's data and disappears along with that data. It is recorded here only so nobody reintroduces the same join-on-a-foreign-id pattern. In v3 a claim is identified by its own id, and a verification backend's identifier is a plugin-owned field it may not repurpose (§2.5).

9.5 URLs

The v2 scheme, `ken.promisetracker.org/william-ruto`, and the v3 scheme, `promisetracker.org/ke/politicians/william-ruto`, share nothing. With no public launch behind it there is no URL history to preserve and no redirect map to build.

One thing to confirm before launch: whether any v2 subdomain has been publicly reachable long enough to be indexed or linked from anywhere. If it has, a redirect map costs little and is worth having. If it has not, which is the expected answer, nothing is owed and the new sitemap is simply the first one.

The single-origin design is worth stating on its own merits, independent of any move. One origin instead of one per country means one TLS certificate, one CDN configuration, one cookie domain, shared caching, and cross-country pages that are possible rather than impossible.

9.6 Launch sequence

Phase	What happens	Exit
0. Stand up	Deploy v3 with Mongo, S3, a domain and monitoring. Boot validation green with `PT_PLUGINS=core` alone	Health endpoint green, and an editor can sign in
1. Seed	Author statuses and groups, categories, the first country, users, site settings	§9.2 complete, and a page renders
2. Editorial pilot	Real documents through the real pipeline: upload, extract, review, publish, with the site not yet public	Editors have published a country's worth of claims and are not asking for workarounds
3. Public launch	Site public, sitemap submitted, submission form open	Traffic served from cache, search works, no errors above baseline
4. Observe	Watch queues, cache hit rate and moderation volume for two weeks	Stable, and the submission backlog is being worked

There is no parallel run, no freeze and no DNS rollback, because there is nothing live to run alongside, freeze or roll back to. The rollback for phase 3 is to not launch yet, which is the cheapest rollback there is, and a good reason not to compress phase 2.

Superdesk is not on this path. It arrives in Phase 3 of the roadmap (§10.1), after launch, by switching `PT_VERIFICATION` from `native`. Claims verified natively before that point keep their verdicts, and the adapter takes over new ones. Launching does not wait on someone else's instance.

9.7 What to verify before going public

1. The system runs with `PT_PLUGINS=core` alone: no Superdesk, no AI, no Tika, no credentials in the environment. If it does not, D4 is broken.
2. Every `claim-status` record has a `group`, and the groups add up to the scorecard an editor expects.
3. Every claim's `currentStatus` is derived from a published update, never typed into a field (§3.5).
4. A French document, uploaded to v3 and processed by v3, renders with correct diacritics. This is the forward test for the fixed extractor (§2.7).
5. An unpublished country hides every claim beneath it, and publishing it reveals them within the revalidation window (§4.3).
6. Claim pages carry valid ClaimReview JSON-LD and canonical URLs, and the sitemap resolves.
7. No claim is publicly visible without a verdict.
8. The Airtable `CSO Promise Updates` backlog has been triaged or dropped on purpose.

---

10. Roadmap

10.1 Phasing

Each phase ends with something usable. Nothing here is a big-bang release.

Phase 1. The core, standing alone

Goal: a complete, publishable PromiseTracker with no external dependency (D4).

Kernel, plugin contract, registry and boot validation. Every collection in §3. `recordStatus`, `_status` and the cascade. Roles. Admin upload and URL connectors. Plaintext and HTML extraction. CSV and XLSX import. The native review queue. The public site with typed routes, caching, pagination, filtering and search. Sitemaps, canonicals, OG images and ClaimReview. The submission form and moderation queue.

Done when a non-developer stands up a country, enters a politician and a promise by hand, adds three dated updates, publishes, and the public page renders the timeline, with no API key anywhere in the environment.

Phase 2. Scale and assist

`plugin-tika` for PDF, Word, Excel and the rest. `plugin-ai` with one provider. The candidate review UI. Maps (D17). Embeds and the public read API. CSV export. The language switcher. Per-country editorial scoping (§4.6).

Country scoping is first in this phase, not last. It is a `where` clause and a field guard with no migration, and it has to exist before a partner newsroom gets an account.

Phase 3. The newsroom

`plugin-superdesk`. Superdesk instance configuration and validation. The HTTP-push receiver and the reconciliation poll. Verdict round-trip. The cutover from `native` to `superdesk` verification.

Superdesk is deliberately last, and deliberately after launch (§9.6). Integrating with a newsroom system that must be configured before the integration can work is the step most likely to slip on someone else's calendar, and nothing before it should wait on that.

Later

Additional LLM adapters. OCR for scanned manifestos. RSS and feed connectors. Automated candidate detection from news. Additional languages. Comparison views across countries. Alerting for citizens following a promise.

10.2 Scale targets

	Phase 1	Phase 2	Phase 3+
Countries	5	20	50+
Actors	200	2,000	10,000
Claims	2,000	25,000	150,000
Claim updates	5,000	75,000	500,000
Documents	500	10,000	50,000
Monthly visitors	20,000	200,000	1,000,000

How each "must not break" is actually satisfied

Stated as a mechanism rather than an aspiration, which is the check §10.4 asks for.

Target	Mechanism
More countries and newsrooms	A country is one record with a code. Creating it yields a landing page, listings and a sitemap with no deploy and no configuration (§7.1). There is no tenancy to provision (D10)
Public traffic spikes	Tag-based caching with precise `revalidateTag` on publish, `generateStaticParams` for published records, CDN-cacheable embeds and API, immutable media headers. The read path has no external dependency (§7.2, §8.8)
Volume of claims and documents	Server-side pagination and filtering with aggregation facets, compound indexes on every listing query, text indexes for search, `depth` specified per query, and a cascade job that batches rather than writing per record (§7.4, §4.3)
Editorial throughput	One indexed `processing.status` as the work queue, a candidate review queue with per-row decisions and attribution, Payload document locking, roles separating contribution from publication, and a moderation queue that never blocks the editorial path (§5.4, §4.5)

10.3 Open questions

Nothing here blocks writing code in Phase 1. Items marked blocking must be resolved before the phase they belong to starts.

Superdesk, all blocking for Phase 3

Every one of these needs a person with access to the instance, not a decision from this document.

1. **Content API prefix, and whether it is enabled.** Derived from `CONTENTAPI_URL`, it may be `/v1`, `/capi` or anything else. Do not hardcode.

2. **Is the Production API deployed?** It is a separate process and many installations skip it. If it is absent, reading unpublished items needs the internal API and a session token.
3. **Content profile id** for claim or fact-check items, and whether one exists yet.
4. **Vocabulary ids and qcodes** for verdict, claim type and category, and whether verdict is modelled as a `subject` scheme, which is queryable, or an `extra` field, which is not indexed by default.
5. **Desk and stage ids** for the fact-checking desk.
6. **Service account** with the `archive` privilege, and the newsroom's agreement to it.
7. **Is `http_push` available,** and can a subscriber and product be configured for PromiseTracker? If not, polling is the only inbound path and latency rises to the poll interval.
8. **Superdesk version.** The changelog stops at 2.4.5 in 2022 and the codebase is mid-rewrite. `content_api/items` still carries a legacy auth shim and a `TODO-ASYNC` on its GET endpoint. Pin the version and re-verify the query surface.
9. **Is `extra.*` mapped in Elasticsearch** on this instance?
10. **Do content-list webhooks exist** in the deployed version? They are on `develop` and not in the published docs.

Product

11. **Verdict vocabulary.** v2 inherits Meedan's `unverified / verified / false / misleading`. Is that the right set for a promise tracker, where the question is usually whether a promise is accurately attributed rather than whether it is true? Worth deciding before the Superdesk vocabularies are created.
12. **Status taxonomy and groups.** The five `ClaimStatus.group` values drive every scorecard. An editor must assign them before the first claim is published (§9.2).
13. **Category taxonomy.** Authored from scratch. v2's free-text values are a starting list, not a source.
14. **Claim types.** Are the four in D2 right, and does each need different fields on the public page?

Technical

15. **Map tile provider** and its licence for commercial and embedded use. Blocking for maps.
16. **Boundary data source,** whether geoBoundaries, GADM or national sources, and its attribution requirements. Blocking for maps.
17. **GeoJSON simplification** step for large boundaries (§3.9).
18. **Search ceiling.** At what volume do MongoDB text indexes stop being adequate, and is Atlas Search available on the hosting plan?
19. **Rewrite or refactor** (D12), a separate technical decision.

10.4 How to review this specification

1. **Walk the v2.0 reference PDF section by section.** Every section should be carried forward, deliberately dropped with a stated reason, or superseded. Flag silent omissions.
2. **Check traceability.** Every "v2 does X" claim should resolve to a real path in the repository. Spot-check `src/tasks/extractPromises.ts:189-213`, `src/tasks/extractDocuments.ts:19`, `src/collections/Promises.ts:12`, `src/plugins/index.ts:93` and `src/lib/security/headers.mjs:53`.
3. **Trace the zero-plugin path on paper:** create country, create actor, upload document, extract, review, publish, public page. If any step needs an external service, D4 is broken.
4. **Trace the Superdesk path** with `PT_VERIFICATION=superdesk`, and confirm every field crossing the boundary appears in the mapping table in §6.2.
5. **Check each scale target** against §10.2 and reject any row that is an aspiration rather than a mechanism.
6. **Argue with the non-goals** in §1.6. They are the cheapest thing to change now and the most expensive later.

10.5 Document history

Version	Date	Change
3.0.0-draft.1	2026-11-06	Initial draft. Supersedes *PromiseTracker System Specification v2.0*, a reference architecture that was never implemented
3.0.0-draft.2	2026-11-09	v2 confirmed never live. §9 rewritten from a v2-to-v3 migration into a launch plan: seeding order, retiring Airtable and Check, launch sequence. No data mapping, cutover or redirect map
3.0.0-draft.3	2026-11-09	Editorial pass across all twelve documents before circulation: plainer sentences, no em dashes, less padding, roughly 10% shorter. Added the review brief as the entry point